PART TWO · 第二部分

Skills: Claude's Extensible Knowledge System

Skills:Claude 可扩展的知识系统

Knowledge that belongs in your head should not stay there. Move it.

"属于你脑子里的知识,不该一直留在那里——把它搬出来。"

CHAPTER FOUR · 第四章

Skills Fundamentals: The SKILL.md Anatomy

Skills 基础:SKILL.md 的解剖

> *"The structure of a thing is the meaning of a thing. Change the structure and you have changed the meaning, whether you intended to or not."*
>
> — Christopher Alexander, paraphrased

"一事之结构,即一事之意义。结构一变,意义便变——无论你是否本意如此。"

—— 克里斯托弗·亚历山大(转述)

← PART ONE 收尾 后一页 →

> This chapter teaches the anatomy of a skill. Not the philosophy, not the strategy, not the high-level reasons skills exist. The anatomy. By the end of this chapter, the reader will be able to look at any `SKILL.md` file in the wild and explain, line by line, what every part of it does, why it is shaped the way it is, and how it will behave when Claude encounters it in a real session. The reader will also be able to write a `SKILL.md` file from a blank page without consulting documentation, because every field, every section, and every convention will be in working memory.

本章讲的是 skill 的*解剖*。不是哲学,不是策略,也不是"skill 之所以存在"的高层理由——是解剖。读完本章,读者应该能够随手拿一份野外的 `SKILL.md` 文件,逐行讲清楚:每一处在做什么、为什么长成那个样子、Claude 在真实 session 中遇到它时,它会如何表现。读者还应该能在一张白纸上,不查任何文档,自己写出一份 `SKILL.md` 文件——因为每一个字段、每一节、每一条惯例,都已经装进了工作记忆。

> Begin with the simplest possible definition. A skill is a folder. The folder lives in either the user's personal skills directory or in the project's skills directory. The folder contains, at minimum, one file named `SKILL.md`. The folder may also contain subfolders for supporting material, but those are optional. The skill's name is the name of the folder. That is the entire structural definition.

从最简定义起步。一个 skill,就是一个文件夹。该文件夹住在两处之一:用户的个人 skill 目录,或者项目的 skill 目录。文件夹里至少要有一个文件,叫 `SKILL.md`;它也可以再含若干子目录用来放支撑材料——但那是可选的。Skill 的名字,就是文件夹的名字。这就是"结构定义"的全部。

> The folder location matters, because location determines scope. A skill placed in the user's personal skills directory, which by convention lives at `~/.claude/skills/` on Unix-like systems, is available in every Claude Code session that user opens, regardless of which project that session is operating on. A skill placed in a project's skills directory, which lives at `.claude/skills/` at the root of that project, is available only when Claude Code is run inside that project. This split exists for an obvious reason. Some skills are personal; they reflect the way a particular developer prefers to work, and they should travel with that developer across every project. Other skills are project-specific; they encode conventions that apply only to a specific codebase, and they should travel with that codebase wherever it goes, including to teammates who clone the repository fresh.

文件夹放在哪里很关键,因为"位置"决定"作用域"。放在用户个人 skill 目录里的 skill——按惯例,在类 Unix 系统上位于 `~/.claude/skills/`——在该用户打开的*每一个* Claude Code session 中都可用,无论那个 session 正在哪个项目上工作。放在项目 skill 目录里的 skill——位于该项目根目录的 `.claude/skills/`——则只在 Claude Code 跑在该项目内时才可用。这种二分法存在的原因显而易见:有些 skill 是个人的,反映的是某位开发者特有的工作偏好,应该跟着这位开发者穿越每一个项目;还有些 skill 是项目专属的,编码的是只适用于某个特定代码库的规约,应该跟着那份代码库走,包括走到任何一位刚刚 clone 仓库的同事手上。

> The folder name becomes the skill name. There are conventions worth following. Skill folder names should be lowercase, hyphenated, and descriptive. A skill that drafts pull request descriptions might live in a folder called `pr-description`. A skill that scaffolds new React components might live in `component-scaffold`. The name is what Claude will see when it loads the skill's metadata at startup, and it is what the developer will see in any tooling that lists installed skills. A clear, short, descriptive name is, in this respect, a small but real act of good engineering.

文件夹名就是 skill 的名字。这里有若干值得遵守的惯例:Skill 的文件夹名应当全小写、用连字符分隔、具描述性。一个负责起草 pull request 描述的 skill,可以放在名为 `pr-description` 的文件夹里;一个负责搭新 React 组件脚手架的 skill,可以放在 `component-scaffold`。这个名,既是 Claude 在启动时加载 skill 元数据所看到的,也是开发者在任何"列出已安装 skill"的工具里会看到的。清晰、简短、具描述性的名字,在这层意义上,本身就是一项小而真实的"好工程"。

The `SKILL.md` File · `SKILL.md` 文件

← Chapter 4 起始

后一页 →

> Inside the folder sits `SKILL.md`. This file has two parts, and the parts are separated by a fence of three hyphens on a line of their own. The first part is YAML frontmatter, which carries structured metadata about the skill. The second part is the markdown body, which carries the instructions Claude will follow when the skill is active. The frontmatter is short. The body is as long as it needs to be, within sensible limits that this chapter will discuss in due course.

`SKILL.md` 住在这个文件夹里。该文件有两部分,两部分之间用一行单独成行的"三个连字符"作为分界栅栏(fence)。第一部分是 YAML frontmatter,携带关于该 skill 的结构化元数据;第二部分是 markdown 正文,携带"该 skill 被激活时 Claude 应遵循的指令"。Frontmatter 很短;正文则在合理限度内,需长则长——本章稍后会讨论这个"合理限度"。

```
---
name: pr-description
description: Drafts a pull request description in the conventional format for this
  team. Use whenever the user asks for a PR description, pull request
  description, PR body, or asks "write a PR for this branch" or similar
  phrasing.
---

# PR Description

When asked to write a pull request description, follow this format exactly:

## Summary

A single sentence that explains what this PR does, written in the imperative
mood.

## What changed

A bulleted list of the concrete changes, one bullet per logical change.

## Testing

A short paragraph describing how the change was tested. If new tests were
added, mention them.

## Risk

A single sentence categorizing risk as low, medium, or high, with one short
reason.

Always read the diff before writing the description. Do not invent changes
that do not appear in the diff.
```

> The example above is a complete, working skill. It has a name. It has a description. It has a body. It can be saved into a folder named `pr-description`, that folder can be placed inside `~/.claude/skills/`, and on the next Claude Code session it will be discoverable. When the user later types something like `give me a PR description for this branch`, Claude will recognize that the skill applies, load its body into context, and produce output that follows the format the body specifies.

上面这份示例,就是一个完整、可工作的 skill。它有名字,有 description,有正文。它可以存进一个名为 `pr-description` 的文件夹,再把这个文件夹放进 `~/.claude/skills/` 里;下一个 Claude Code session,这个 skill 就是可被发现的了。此刻,用户再敲一行类似 `give me a PR description for this branch` 的请求,Claude 就会识别出"这个 skill 用得上",把它的正文加载进上下文,然后产出严格按"该正文"规定格式的输出。

[注:代码块原文保留,以下为本页中译补充]

上面 YAML frontmatter 的两个字段,分别为:① `name`(skill 的内部标识,与文件夹同名);② `description`(给 Claude 看的触发说明,决定 skill 在何种用户请求下被激活)。正文则是一段简洁的指令,Claude 加载该 skill 后,会逐字遵循其中规定的格式。整段示例的"麻雀虽小、五脏俱全",是后面所有 skill 的最小模板。

← Chapter 4 上一页

后一页 →

Look closely at the frontmatter. There are exactly two required fields. The first is `name`, which must match the folder name and is what Claude uses to refer to the skill internally. The second is `description`, which is the field that determines whether and when the skill is invoked. The description is the most important field in the entire file, and the next chapter is devoted entirely to writing it well, because no other single decision affects skill behavior more.

把 frontmatter 凑近看:里面恰好只有两个必填字段。第一个是 `name` ——它必须与文件夹同名,也是 Claude 内部指代该 skill 时所使用的标识。第二个是 `description` ——决定该 skill 是否被调用、以及在何时被调用的字段。`description` 是整份文件里最重要的字段,下一章会整章专门讲"如何把它写好"——因为再没有其他任何单点决策,能比它对 skill 行为的影响更大。

There is also an optional third field worth knowing about now: `allowed-tools`. This field, when present, restricts the set of tools Claude is permitted to use while operating under this skill. A skill that exists only to compose written prose, for example, might restrict tools to `Read` and `Grep` and exclude `Edit`, `Write`, and `Bash`, so that the act of using the skill cannot accidentally modify the file system. The `allowed-tools` field is a Claude Code feature; when skills are used through the Anthropic API or the SDK, tool access is controlled at the application level instead. For most readers of this book, who are running skills inside Claude Code, `allowed-tools` is a useful safety mechanism that should be reached for whenever a skill has no legitimate reason to write to disk or run shell commands.

还有第三个可选字段,值得现在提一句:`allowed-tools`。这个字段如果出现,会限制"该 skill 处于激活态时"Claude 允许使用的工具集。举例来说,一个只用来组织书面文字的 skill,可以把工具集限制成 `Read` 与 `Grep`,而把 `Edit`、`Write`、`Bash` 排除掉——这样"使用这个 skill"这个动作本身,就不可能意外地改动文件系统。`allowed-tools` 是 Claude Code 的特性;当 skill 走 Anthropic API 或 SDK 时,工具访问由应用层控制。本书大多数读者,都是在 Claude Code 里跑 skill;在那种场景下,`allowed-tools` 是一道好用的安全闸,凡是一个 skill 没有正当理由去写盘或跑 shell 命令时,都该把这条闸拉上。

A Worked Example · 一个完整的具例

A second example, slightly larger, will demonstrate `allowed-tools` in context.

下面这份稍大一点的第二个示例,会演示 `allowed-tools` 在具体语境中的样子。

```
---
name: explain-codebase
description: Produces a high-level architectural summary of a codebase using
  only read operations. Use when the user asks "explain this codebase,"
  "give me a tour of this repo," "what does this project do," or any
  request to understand a codebase without modifying it.
allowed-tools: Read, Grep, Glob
```

```
---

# Explain Codebase

When the user asks for an explanation of a codebase, follow this procedure:

1. Read the README and any top-level documentation files first.
2. List the top-level directories and infer the architectural pattern from
   their names.
3. Read package.json or the equivalent dependency manifest to understand the
   technology stack.
4. Identify the entry point of the application by checking the manifest for
   a "main" or "bin" field.
5. Read the entry point file and trace the first one or two layers of
   imports.
6. Produce a summary with these sections:
```

> *[注:代码块原文保留,以下为本页中译补充]*

这个示例的 `description` 写得相当考究——它既说明了"该 skill 干什么",又用一长串具体表述把"用户可能会怎么问"列得很齐:explain this codebase、give me a tour of this repo、what does this project do……Claude 据此判断"何时该激活"。`allowed-tools: Read, Grep, Glob` 三件,都是只读类工具,刻意把 Edit、Write、Bash 排除在外——这就从机制上保证了"explain-codebase"这个 skill 在跑的时候,绝对不会改你任何一个文件。第 6 步"Produce a summary with these sections:"会在下一页接续展开。

← Chapter 4 上一页

后一页 →


```
6. Produce a summary with these sections:
   - Stack
   - Architecture
   - Entry point and request flow
   - Notable patterns or conventions
   - Open questions or things you would need to read more to understand

Never modify any files while operating under this skill. The allowed-tools
restriction enforces this, but the instruction is repeated for clarity.
```

> *[注:上一行起、到此行为上一页示例 SKILL.md 的续写。代码块原文保留,以下为本页中译补充]*

上面这段,是上一页那份 `explain-codebase` skill 正文的剩余部分。第 6 步要求"按以下小节给出总结"——Stack、Architecture、Entry point and request flow、Notable patterns or conventions、Open questions or things you would need to read more to understand——这种"固定小节"结构让 Claude 每次产出时都按相同骨架,既方便人对比、也方便人检索。最后那句"Never modify any files"与 frontmatter 中的 `allowed-tools` 形成*双保险*:即使配置层被人不小心改坏,正文里的明确指令也仍然把"只读"这件事再讲一遍。

> The reader should pause and notice what just happened. With twenty-five lines of plain text in a folder, the reader's Claude Code installation has been taught a procedure that is now repeatable, consistent, and safe. The next time the reader joins a new project and asks Claude to explain the codebase, the same procedure will run, in the same order, with the same output structure, and Claude will be physically incapable of writing to any file while doing so. This is the leverage skills offer. A small, careful piece of configuration produces a substantial and ongoing improvement in the developer's daily experience.

请读者在此停一拍,留意刚才发生了什么:一个文件夹、二十五行纯文本,就把"一套可重复、可一致、可安全"的工作流,教给了读者这台 Claude Code 安装。下一次读者加入新项目、让 Claude 解释代码库时,同一套流程将以同一次序、同一种输出结构跑出来;而在此期间,Claude 在物理上*根本不可能*向任何文件写入。这就是 skill 所给出的杠杆。一小份、用心写就的配置,就能给开发者的日常体验带来实质性、可持续的提升。

> The body of a skill is not free-form, but it is flexible. There is no required template. Headings, lists, examples, code samples, references to other files, are all allowed. The two principles that govern good skill body design are clarity and concision. Clarity, because Claude is going to follow the body's instructions literally, and ambiguity in the body produces ambiguity in the output. Concision, because the body counts against the context window every time the skill activates, and a verbose body wastes tokens that could be spent on the actual work.

Skill 的正文,不是自由格式,但很灵活:并没有强制模板。标题、列表、示例、代码片段、到其他文件的引用——都允许。指导"好的 skill 正文设计"的两条原则,是*清晰*与*简洁*。清晰,因为 Claude 会逐字遵循正文里的指令,正文里的歧义,会原封不动地变成输出里的歧义。简洁,因为正文每次激活时都要进上下文窗口,冗长的正文是在浪费那些本可以花在"真正的工作"上的 token。

> A useful rule of thumb is that a `SKILL.md` body should stay under five hundred lines. If a skill seems to want more than that, the right move is almost always to factor the surplus content into separate files within the skill folder and reference them from the body. A skill folder by convention may contain three kinds of supporting material, organized into three subfolders. The `scripts` folder contains executable code that Claude can run as part of the skill. The `references` folder contains additional documentation that Claude should read when relevant, but that does not need to live in `SKILL.md`. The `assets` folder contains files that Claude will use as templates or fixtures during the skill's execution. These three subfolders are conventions established by Anthropic in the official skills repository, and

一条好用的经验法则是:`SKILL.md` 的正文应当控制在五百行以内。如果一个 skill 看起来要超过这个量,几乎总是应当把多出来的内容拆成 skill 文件夹内的若干独立文件,再由正文去引用它们。一个 skill 文件夹按惯例可以容纳三类支撑材料,各自放在三个子目录里:`scripts` 目录,装可执行代码——Claude 在该 skill 执行过程中可以运行;`references` 目录,装额外文档——Claude 在相关时阅读,但不必常年住在 `SKILL.md` 里;`assets` 目录,装那些会被 Claude 当作模板或夹具使用的文件。这三个子目录是 Anthropic 在官方 skill 仓库里确立的惯例,

← Chapter 4 上一页

后一页 →

> following them makes the skill folder predictable to read for anyone who joins later, including future versions of the developer who wrote it.

遵守这套惯例,会让 skill 文件夹对"之后加入的任何人"——包括将来某一天再回看自己代码的那位开发者本人——都保持一种"可预期的"读法。

> The progressive disclosure principle, mentioned in the previous chapter, is what makes the bundled-files pattern practical. When Claude starts a session, it loads only the descriptions of available skills, not their bodies. When Claude decides that a specific skill applies, it loads that skill's `SKILL.md` body into context. Only at that point, if the body references a file in `references` or asks for the execution of a script in `scripts`, does Claude reach further into the skill folder to pick up what is needed. The total token cost of having forty skills installed, none of which is currently relevant, is roughly the cost of forty short descriptions. The cost only rises when a skill becomes relevant, and even then, the rise is bounded by what the body and any referenced files actually contain.

上一章提到过的"渐进式披露"(progressive disclosure)原则,正是让"打包文件"这种模式得以成立的关键。Claude 启动一个 session 时,只加载"可用 skill 的描述",不加载它们各自的正文。Claude 决定某个具体 skill 适用时,才会把该 skill 的 `SKILL.md` 正文加载进上下文。再接下来——也只有到了那一步——若正文引用了 `references` 里的某份文件,或要求执行 `scripts` 里的某段脚本,Claude 才会再往 skill 文件夹深处去取所需之物。一个 session 上"装了四十个 skill、但此刻一个都用不上"的代价,大致就是"四十条短描述"的总和;代价要"上涨",得等到某个 skill 真的变得相关;而即便上涨,其幅度也被正文及所引用文件所实际包含的内容所限。

Skills as a System · Skills 作为一套系统

> Take a moment to consider the architecture from this angle. The skill system is a three-layer cache. The first layer, always loaded, is the metadata: name and description for every skill. The second layer, loaded on demand when the skill activates, is the `SKILL.md` body. The third layer, loaded only when the body asks for it, is whatever lives in `scripts`, `references`, and `assets`. Each layer costs more tokens than the one above it, and each layer is reached only when the layer above has decided that more detail is required. This cascade is what allows a Claude Code installation to host a substantial library of skills without paying a substantial token tax for them.

请换一个角度,看看这套架构。Skill 系统,其实是一座三层缓存:第一层——常驻加载——是元数据:每个 skill 的 `name` 与 `description`。第二层——按需加载,在 skill 激活时——是 `SKILL.md` 的正文。第三层——仅在正文索取时加载——是 `scripts`、`references`、`assets` 这三个目录里住着的东西。每一层的 token 代价,都比上一层更高;而每一层都只在"上一层已判定需要更多细节"时才会被触及。这道级联,正是"在一台 Claude Code 安装上挂一整个颇具规模的 skill 库,却不必为它付出巨额 token 税"的原因所在。

> There is a final aspect of skill anatomy that deserves attention before this chapter closes, and it concerns the relationship between skills and Claude Code's older slash-command system. Slash commands are the explicit, user-invoked counterpart to skills. A user types a slash followed by a command name, and Claude executes the corresponding command. Slash commands have existed in Claude Code longer than skills have, and many developers have built up libraries of them. As of recent updates, slash commands and skills have been unified at the implementation level, which means that a file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both register a `/deploy` slash command and behave the same way when invoked explicitly. The practical

在本章收尾之前,skill 解剖中还有最后一个面向值得提上一笔,它涉及"skill"与"Claude Code 沿用已久的 slash-command 系统"之间的关系。Slash command 是 skill 那种"显式、由用户发起"的对应物:用户敲一个斜杠,后面跟命令名,Claude 就会去执行相应的命令。Slash command 在 Claude Code 里的历史比 skill 更久,许多开发者已经围绕它们攒起了不少命令库。在最近一次更新之后,slash command 与 skill 已经在实现层上被统一了——也就是说,放在 `.claude/commands/deploy.md` 的文件,与放在 `.claude/skills/deploy/SKILL.md` 的 skill,会注册出同一个 `/deploy` slash command,被显式调用时行为一致。落到实操——

← Chapter 4 上一页

后一页 →

> recommendation that emerges from this convergence is to prefer the skill format for any new work, because skills support the full feature set, including bundled files and automatic invocation, whereas the older command format is a subset.

——从这种"合流"中浮现的实操建议是:任何新工作,都该优先采用 skill 格式。因为 skill 支持完整的功能集,包括"打包文件"与"自动调用";而更老的 command 格式,只是其一个子集。

> Automatic invocation is the property that makes skills feel different from slash commands in daily use. A slash command requires the user to remember the command and to type it. A skill, when its description is well written, requires nothing of the user except that she describe her task in natural language. Claude reads the descriptions, recognizes which skills apply, and activates them silently. The user does not need to know which skills exist or which is firing at any given moment. The environment simply gets better at the user's requests, in ways that match the configuration the developer has installed. This silent, automatic activation is the property that makes skills, in the words of one prominent commentator, possibly a bigger deal than MCP. The barrier between intention and capability collapses to zero. A user describes what she wants, and the right capability arrives on its own.

"自动调用"这一性质,正是 skill 在日常使用中与 slash command 体感截然不同的根源。Slash command 要求用户记得那条命令、并且亲手敲出来;而 skill,只要 description 写得到位,对用户的要求就只剩一条——用自然语言描述她的任务。Claude 会去读各条 description,识别哪些 skill 适用,再静悄悄地把它们激活。用户完全不需要知道"装了哪些 skill",也不需要知道"此刻到底是哪一条在跑"。环境会变得"对用户的请求越来越合手",而这种合手,正贴合开发者所装入的那份配置。这种"无声的、自动的激活",才是某位知名评论者口中"skill 也许比 MCP 更具分量"那一判断的依据。*意图与能力*之间的那道墙,被压到近乎为零——用户只需说出她想要什么,合适的能力就会自行到位。

> Before this chapter closes, a third worked example is worth examining, because it shows the anatomy at its most expressive. The previous two examples were small. Real skills, the ones that earn their place in a working library, often carry a little more weight. The example below is a skill called `sql-review`, and its job is to review a SQL query for performance, correctness, and security issues before it goes anywhere near a production database.

在本章收尾之前,值得再看一份"第三个完整的具例"——因为它会把 skill 的解剖展示到最具表现力的那种程度。前两份示例都比较小。真正在工作的 skill 库里能占有一席之地的那批,往往要再重一些。下面这份示例的 skill 叫 `sql-review`——它的职责,是在一条 SQL 查询逼近任何生产数据库之前,先把它在性能、正确性、安全性上各做一轮 review。

```
---
name: sql-review
description: Reviews a SQL query for performance, correctness, and security
  issues. Use whenever the user pastes a SQL query and asks "review this
  query," "is this efficient," "will this scale," "any issues with this
  SQL," or describes wanting feedback on a query before running it. This
  skill is for review only; it does not execute the query.
allowed-tools: Read
---

# SQL Review

When asked to review a SQL query, follow this procedure exactly.

## Step 1: Read the query carefully
```

> *[注:代码块原文保留,以下为本页中译补充]*

注意这份示例的 `description` ——它先讲清"做什么"(review SQL),再列出一长串"用户在自然语言里可能怎么问"(review this query / is this efficient / will this scale / any issues with this SQL / 任何"在跑之前想听听反馈"的请求),最后特意加了一句"本 skill 仅做 review,不会执行查询"。这种"主动划清边界"的写法,值得每个写 skill 的人借鉴:与其让 Claude 误判激活时机,不如让 description 自己先把这层意图讲清。`allowed-tools: Read` ——只读,与该 skill 的"纯 review"职责严丝合缝。第 1 步"仔细读 query"会在下一页展开。

← Chapter 4 上一页

后一页 →


```
Read every clause. Identify the tables touched, the join conditions, the
filter conditions, the projection, and any subqueries or CTEs.

## Step 2: Performance review

Check for:

- Missing or non-sargable WHERE clauses (functions wrapping indexed columns)
- Joins on non-indexed columns
- SELECT * in production paths
- Cartesian products from missing or wrong join conditions
- Subqueries that could be CTEs or joins for clarity and performance
- Aggregations over large tables without filters

## Step 3: Correctness review

Check for:

- NULL handling, especially in joins and aggregations
- Off-by-one errors in BETWEEN and date ranges
- Implicit type coercion that could mask bugs
- LIMIT without ORDER BY, which produces nondeterministic results

## Step 4: Security review

Check for:

- String concatenation building queries (SQL injection vector)
- Missing parameterization where user input is involved
- Permissions assumed but not verified

## Step 5: Output format

Produce the review in this format:

### Verdict

One sentence: safe to run, run with caution, or do not run.

### Performance

A short bulleted list of issues, each with a one-line explanation and a
one-line fix.

### Correctness
```

```
Same format.

### Security

Same format.
```

[注:代码块原文保留,以下为本页中译补充]

整份 `sql-review` 的正文,展示了"多步骤、严格流程型"skill 的典型写法:Step 1 把 query 读透——把表、join 条件、过滤条件、投影、子查询/CTE 都标出来;Step 2 在性能维度做检查清单(sargable、索引覆盖、SELECT *、笛卡尔积等);Step 3 在正确性维度做检查清单(NULL 处理、BETWEEN 边界、隐式类型转换、LIMIT 不带 ORDER BY 等);Step 4 在安全性维度做检查清单(字符串拼接、未参数化、未核实权限);Step 5 固定输出格式——Verdict / Performance / Correctness / Security 四个小节,每个小节都"一句解释 + 一句修复"。这种"流程清晰、检查清单可枚举、输出格式固定"的写法,是把"人类专家的复盘经验"装进 Claude 的标准做法,值得在所有复杂 skill 上照搬。

← Chapter 4 上一页

后一页 →

Multi-File Skills · 多文件 Skill

> That example deserves a moment of attention. It is twice as long as the earlier examples, and the additional length is doing real work. The body is broken into named steps, each step has a focused checklist, and the output format is specified down to the section headings. A skill written this way produces strikingly consistent output, because almost every degree of freedom that could produce variability has been pinned down. The query is read in the same order every time. The same five performance issues are looked for in the same order. The output is structured the same way every time. The reader who runs this skill against ten different queries over the course of a week will get ten reviews that feel like they came from the same careful reviewer, because in a meaningful sense they did.

那份示例值得停一拍来品味。它比前面的示例长了一倍,而那多出来的长度,确实在干实事:正文被切成若干"已命名的步骤",每一步都带一份聚焦的检查清单,输出格式被规定到小节标题这一颗粒度。这样写出来的 skill,产出的输出会有一种"惊人的一致"——因为几乎每一个可能造成差异的自由度,都被钉死了。Query 每次都被以相同的次序去读;那五条性能问题每次都被以相同的次序去查;输出每次都被按相同的结构来组织。读者一星期内拿这条 skill 跑十条不同的 query,会得到十份"看起来出自同一位谨慎 reviewer 之手"的 review——而从某种意义上讲,也确实如此。

> This is a property worth dwelling on. Skills do not merely save typing. They produce consistency. A developer working alone can have ten different commit messages on a Monday and forty different commit messages by Friday, all of them written by her, none of them following the same internal logic. The same developer with a commit-message skill installed will have one hundred commit messages by the end of the month that all follow the same format, because the format is no longer carried by her attention but by the skill. Consistency, in this sense, is a separate benefit from speed, and it compounds in a different way. Speed saves time per commit. Consistency makes the entire history of commits more useful, because future readers can rely on the format being stable.

这一性质值得多停留一会儿。Skill 不只是省打字——它产出*一致性*。一个独自干活的开发者,周一可能写出十种不同风格的 commit message,到周五已经积出四十种,全是她写的,却没一份遵守同一条内在逻辑。同一开发者一旦装上 commit-message skill,到月底她会写出一百条 commit message,全部遵循同一格式——因为那套格式不再由她的注意力承载,而是由 skill 承载。一致性,在这种意义上,

是"速度"之外另一份独立收益,并且以另一种方式复利。速度省下的是每条 commit 的时间;一致性则让整段 commit 历史都变得更可用,因为未来的读者可以放心地"假定格式是稳定的"。

> Two more anatomical details deserve attention before this chapter closes, because they come up often in real skill libraries and are worth knowing the first time the reader encounters them. The first detail concerns the relationship between the description and the body. A common confusion among new skill authors is the temptation to repeat in the body what the description already says. This is a waste of context, because the description

本章收尾之前,还有两条"解剖层面的细节"值得提一笔,因为它们在真实的 skill 库里频繁出现,值得读者第一次撞见时就了然于胸。第一条,关于 description 与正文之间的关系。Skill 作者新手上路时,常犯的一个误区,就是把 description 已经讲过的话,在正文里再复述一遍——这是对上下文的浪费,因为 description 在该 skill 被"考虑是否激活"时,本来就已经——

```
### Suggested rewrite

If issues are significant, provide a rewritten query in a fenced code block.
If the original is fine, omit this section.

Do not invent issues. If the query is good, say so plainly. A short clean
review is better than a padded one.
```

> *[注:代码块原文保留,以下为本页中译补充]*

上面是 `sql-review` skill 的最后一段正文——新增了"### Suggested rewrite"小节,只有在问题显著时才给出"重写后的 query",并且以一句金句收尾:"不要无中生有。Query 没问题就直说;一份干净简短的 review,胜过一份注水的 review。"这正是把人类 reviewer 的职业操守——"克制"——写进 skill 的好例子。

← Chapter 4 上一页

后一页 →

> is already loaded into context whenever the skill is being considered. The body should pick up where the description leaves off. The description tells Claude when to fire. The body tells Claude what to do once it has fired. Treating the two as having distinct jobs prevents the kind of bloated `SKILL.md` file in which the first paragraph of the body is a restatement of the description.

被加载进上下文了。正文应当从 description 停下的地方继续往下接。Description 告诉 Claude 何时激活;正文告诉 Claude 激活之后做什么。把这两件事视作不同分工,就能避免那种"开篇第一段只是复述 description"的臃肿 `SKILL.md`。

> The second detail concerns the cost of having many skills. A natural worry is that installing fifty skills will somehow degrade the performance of every Claude Code session. In practice, the cost is bounded by progressive disclosure. Fifty skills means fifty descriptions loaded at startup. A typical description is two or three sentences, perhaps thirty to fifty words. Fifty descriptions is therefore on the order of two thousand words, or roughly three thousand tokens. That is a real cost, but it is a small one compared to the size of a modern context window, and it is paid once at session start rather than on every prompt. The body of any specific skill is loaded only when that skill activates, and only one or two skills typically activate per turn. The cost of skill ownership scales gracefully. A library of fifty good skills is not noticeably slower than a library of five.

第二条,关于"装很多 skill"的代价。一个很自然的担心是:装五十个 skill,会不会拖累每一个 Claude Code session 的表现?在实践中,这个代价被"渐进式披露"压在了很小的范围内。五十个 skill,意味着启动时加载五十条 description;一条典型的 description 是两到三句、约三五十个词;五十条合起来,大致是两千个词、约三千 token。这是一笔真实开销,但相对于现代上下文窗口的容量,它只是很小的一笔,且只需在 session 启动时一次性付清,而不是每条 prompt 都要付一次。任何具体 skill 的正文,只在该 skill 激活时才加载;而每轮通常只有一两个 skill 会激活。"拥有 skill 的成本"是优雅地扩展的——五十个好 skill 的库,与五个好 skill 的库,在体感上几乎察觉不到差异。

> This is the anatomy of a skill, complete enough to begin building. A folder, named after the skill, in either the user or project skills directory. A `SKILL.md` file at the root of that folder, with frontmatter declaring at minimum a name and a description, optionally restricting tools, and a markdown body containing the instructions. Optional subfolders for `scripts`, `references`, and `assets`, picked up by Claude only when the body refers to them. A clear, automatic activation model that does not require the user to remember anything. With this anatomy in working memory, the reader is ready for the chapter that follows, in which a real skill is built from the empty folder forward, with every decision made explicit and every common mistake catalogued. The work begins now.

到这里,skill 的解剖就讲完了——足够让读者动手开始搭建:一个文件夹,以 skill 命名,放在 user 或 project 的 skills 目录里;一份 `SKILL.md` 文件,位于该文件夹根目录,frontmatter 至少声明 `name` 与 `description`,可选地限制 tools,正文是 markdown 写成的指令;可选的 `scripts`、`references`、`assets` 子目录,仅在正文引用时被取用;一套清晰、自动的激活模型,不需要用户去"记"什么。把这份解剖装进工作记忆之后,读者就为下一章做好了准备——下一章,会从一张"空文件夹"开始,正向地把一份真正的 skill 搭出来;每一步决策都讲透,每一处常见错都收齐。活,从这里正式开干。

← Chapter 4 上一页

CHAPTER FIVE →

CHAPTER FIVE · 第五章
Writing Your First Skill: A Hands-On Build
亲手写出你的第一个 Skill

> *"The shortest distance between a developer and a working system is the smallest example that ships."*
>
> — Folk wisdom of the open source world

"从开发者到一套能跑的系统,最短的距离,是那份'小到刚刚能发出来'的示例。"

—— 开源世界的民间智慧

> This chapter is a build. By the end of it, the reader will have a working skill installed in her own Claude Code environment, will have used it in a real session, and will have debugged the most common failure mode skills exhibit. The build is small on purpose. The point is not the impressiveness of the example. The point is that every step of the path from empty directory to working skill becomes familiar, so that subsequent skills, larger and more sophisticated, can be built without having to relearn any of it.

这一章,就是一次"搭"。读完之后,读者会有一份"已安装、跑得起来"的 skill,放在她自己的 Claude Code 环境里;会在一次真实 session 中用过它;会亲手 debug 过 skill 最常出现的那一种失败模式。这个搭出来的 skill 故意做得很小。重点不在"示例有多炫",重点在于——

从"空目录"到"能跑的 skill"这整条路上的每一步,都要变得熟悉;这样以后再搭更大、更复杂的 skill 时,就不用把每一步都重新学一遍。

> The skill being built in this chapter is called `commit-message`. Its job is to take the staged changes in a git repository and produce a single commit message in the conventional commits format. This is a skill that almost every developer would benefit from, because almost every developer has, at some point, committed a change with a message that her future self struggled to interpret. The skill will fix this once and for all, by ensuring that every commit message Claude writes follows a consistent format, includes a meaningful summary, and surfaces any breaking changes prominently.

本章要搭的 skill 叫 `commit-message`。它的职责是:拿到一个 git 仓库里的暂存改动,产出一条按 conventional commits 规范写成的 commit message。这是一条几乎每位开发者都用得上的 skill——因为几乎每位开发者都曾在某个时刻,提交过一条"日后回看、连自己都看不懂"的 commit message。这条 skill 会把这件事一劳永逸地治好:Claude 写出的每一条 commit message,都遵守同一格式、都带一句有意义的摘要、并且把任何 breaking change 显著地摆在显眼位置。

> Before opening any files, do the architectural thinking. The decision is, first, where the skill should live. This is a skill that the developer will want available in every project, not only in one. Therefore the skill belongs in the personal skills directory, at `~/.claude/skills/`. The decision is, second, what the skill needs to be able to do. To inspect staged changes, the skill needs the ability to run shell commands, specifically `git diff`. To compose the message, the skill needs to be able to think about the diff. It does not need to write to any file directly, because the user will take the produced message and pass it to `git commit` herself. Therefore the `allowed-tools` field will include `Bash` but not `Edit` or `Write`.

动手开文件之前,先在脑子里把"架构"想清楚。第一项决策,放在哪里?这是一条"开发者希望在每个项目里都用得上"的 skill,而不只是某一个项目。因此,它该住在*个人* skill 目录里,也就是 `~/.claude/skills/`。第二项决策,它需要具备哪些能力?要审阅暂存改动,它得有跑 shell 命令的能力——具体说,是 `git diff`。要把 commit message 组出来,它得有"对 diff 动脑"的能力。它并不需要直接写任何文件——因为用户会把生成的 message 拿去自己跑 `git commit`。因此 `allowed-tools` 字段里,会带上 `Bash`,但*不*带 `Edit`、`Write`。

← Chapter 4 结尾

后一页 →

> The decision is, third, what the description should say. This is where novice skill authors most often fail, because the description has two jobs that pull in opposite directions. It must be specific enough that Claude only fires the skill when it genuinely applies, and it must be broad enough to fire on the variety of phrasings real users actually employ. A description that is too narrow will fail to fire on requests that should clearly trigger it. A description that is too broad will fire when it should not. The next chapter is devoted to the craft of description writing in depth, but for this first build, a reasonable working approach is to draft a description that lists the canonical request, then enumerates a few alternative phrasings the user might use, and finally states what kind of input the skill expects to be working from.

第三项决策,description 该怎么写?这正是 skill 写手新手最容易翻车的地方——因为 description 同时扛着两份彼此拉扯的职责:它必须*足够具体*,Claude 才不会在不该激活的时候乱激活;又必须*足够宽泛*,才能在真实用户千变万化的措辞下都触发得起来。写得太窄,会漏掉那些"明明该触发"的请求;写得太宽,会在不该触发时乱触发。下一章会深入讲 description 写作的手艺;不过就本章这次"首次搭建"而言,一个够用的工作方法是:先写一句"典型请求",再列举几条"用户可能换用的措辞",最后说明本 skill 期望自己拿到的是哪一类输入。

Setting Up the Folder · 创建文件夹

> With these decisions made, the build can begin. Open a terminal. Create the folder.

这些决策一旦落定,搭建就可以开始了。打开终端,把文件夹建出来。

> Inside that folder, create a file named `SKILL.md`. The file will contain frontmatter, then a body. Open the file in any editor and type the following.

在那个文件夹里,新建一个名为 `SKILL.md` 的文件。文件包含 frontmatter,再接一段正文。用任何编辑器打开,敲入下面这些内容。

```
mkdir -p ~/.claude/skills/commit-message
cd ~/.claude/skills/commit-message
```

```
---
name: commit-message
description: Generates a conventional commits message from the currently
  staged changes in a git repository. Use whenever the user asks for a
  commit message, asks to "write a commit," asks "what should I commit
  this as," or otherwise indicates they have staged changes and want a
  message written. Always inspect the diff before writing the message.
allowed-tools: Bash, Read
---

# Commit Message

When asked to produce a commit message, follow this procedure exactly.

## Step 1: Inspect the staged changes

Run `git diff --cached` to see what is currently staged. If nothing is
staged, run `git diff` to see unstaged changes and tell the user that no
changes are staged but here is what would be committed if they were. Do
not proceed to write a message until you have actually read the diff.
```

> *[注:代码块原文保留,以下为本页中译补充]*

第一段是 shell 命令:`mkdir -p ~/.claude/skills/commit-message` 建文件夹,接着 `cd` 进去。第二段是 `SKILL.md` 的内容。注意:① `description` 段在 "Generates a conventional commits message..." 之后,枚举了一长串用户可能用的措辞(a commit message / write a commit / what should I commit this as),并在末尾强调"Always inspect the diff before writing the message"——把"先读 diff、再写 message"这条铁律写死在 description 里;② `allowed-tools: Bash, Read` 正好对应"读 diff + 跑 git 命令"的需求;③ 正文"## Step 1"先硬性规定:必须先看 diff,没 staged 也要明示用户,绝不允许在没读 diff 的情况下动手写 message——这是对"用 skill 之前先把数据看清"这条职业操守的具体化。

```
## Step 2: Choose the type

The commit type is the first part of a conventional commit and must be one
of the following:

- feat: a new feature visible to the user
- fix: a bug fix
- refactor: a change in code that does not alter behavior
- docs: documentation only
- test: tests only
- chore: build tooling, dependencies, configuration
- perf: a change focused on performance
- style: formatting only, no code change

Choose the most specific type that fits. If a single commit spans multiple
types, choose the most user-visible one and mention the others in the body.

## Step 3: Choose the scope

The scope is an optional one-word noun in parentheses indicating which
area of the codebase changed. Examples: feat(auth), fix(api),
refactor(parser). Use a scope when the diff is concentrated in a
recognizable area. Omit it when changes are diffuse.

## Step 4: Write the summary

The summary is the rest of the first line. Keep it under 72 characters.
Use the imperative mood: "add user authentication," not "added user
authentication." Do not capitalize. Do not end with a period.

## Step 5: Write the body, if needed

If the change is non-trivial, add a blank line after the summary, then
write a short paragraph explaining what changed and why. Keep it under
five lines. Skip the body for trivial changes.

## Step 6: Mark breaking changes

If the diff contains anything that breaks an existing API or changes
default behavior, add a line at the bottom that begins with
`BREAKING CHANGE:` followed by an explanation. This line is required for
breaking changes and must not be omitted.

## Step 7: Output

Output the final commit message in a single fenced code block, with no
commentary before or after. The user will copy it directly into git
commit.

## Examples
```

> *[注:代码块原文保留,以下为本页中译补充]*

第 2 步——选 type:从 feat / fix / refactor / docs / test / chore / perf / style 八种里挑最贴切的一种;若一种 commit 跨多类,选"对用户最可见"的那种,其余的写进 body。第 3 步——选 scope:用括号里的一词名词,标出代码库的哪一块被改;只改一处明确区域时给,跨多处则省。第 4 步——写 summary:第一行 72 字符以内,imperative mood(动词原形起头)、不大写、不以句号收尾。第 5 步——写 body(可选):非琐碎改动加一段简短说明,五行以内。第 6 步——breaking change:若有破坏既有 API 或默认行为的改动,文末必须加一行 `BREAKING CHANGE:` 起头,这一行*不得省略*。第 7 步——输出:整条 commit message 用一个 fenced code block 输出,前后不附任何

评注;用户会原样复制到 `git commit`。第 8 步 "## Examples" 会在下一页接续展开。这一整段正文,示范了"把一个公认规范(Conventional Commits)整体编码进 skill"的标准动作:每一步都给具体规则,把"判断空间"压到最小,只留"读懂 diff"那一处真正需要判断的地方。

← Chapter 5 上一页

后一页 →


````
A good message:

```
feat(auth): add password reset endpoint

Adds POST /auth/reset that accepts an email and sends a reset link.
Token expires in one hour. Tested with Postman.
```

A bad message that this skill must avoid:

```
fixed stuff
```
````

> *[注:代码块原文保留,以下为本页中译补充]*

这两段"对照示例",是 `commit-message` skill 正文末尾"## Examples"小节的实际内容:第一条是一份"好 message"——type (feat)、scope (auth)、72 字符内 imperative mood 的 summary、加上不超过五行的 body,完整示范了 conventional commits 规范;第二条是"必须避开的坏 message"—— `fixed stuff`,四个字,既无 type、也无 scope、更没 body。把这种"好/坏"对照放进 skill,等于把人类 reviewer 心里那杆秤,显式地摆到了 Claude 面前。

> Save the file. The skill is now installed. To verify, restart Claude Code so that the new skill is discovered at the next startup. Open Claude Code in any git repository where some changes are staged. Type, in the most natural phrasing the developer can think of, something like `give me a commit message for this`. Claude should recognize that the `commit-message` skill applies, run `git diff --cached` on its own, read the diff, and produce a commit message that follows the seven-step procedure laid out in the body. The whole interaction should feel as if the developer has briefed a careful collaborator, once, and the collaborator now silently applies the same procedure every time the request is made.

保存文件。Skill 这就装上了。要验证它,得重启 Claude Code,让新 skill 在下次启动时被扫到。在任何一个已经有暂存改动的 git 仓库里打开 Claude Code,用开发者能想到的最自然的措辞,敲下类似 `give me a commit message for this` 的话。Claude 应当识别出 `commit-message` skill 适用,自己跑去 `git diff --cached`,读完 diff,然后按正文里七步流程产出一条 commit message。整段交互,看起来就像是开发者向一位谨慎的协作者做了一次情况通报,从此那位协作者每次接到类似请求,都会默默按同一份流程办。

> The first time a developer builds a skill, the most common failure mode is not what one might expect. The failure is not that the skill misbehaves. The failure is that the skill does not fire at all. The developer types a request that should clearly activate the skill, and Claude proceeds to write a commit message in some other format, ignoring the skill entirely. This failure has one cause in roughly nine cases out of ten, and the cause is the description.

开发者第一次搭 skill 时,最常碰到的失败模式,并不是大家预想的那种。失败不是"skill 行为不对",而是"skill 干脆不触发"——开发者敲了一段本应明确激活该 skill 的请求,Claude 直接用别的格式写出了一条 commit message,完全无视这份 skill。这种失败,十有八九只有一个原因,而那个原因,就是 description。

Failure Modes and Fixes · 失败模式与修法

> Walk through what is happening when the failure occurs. At session start, Claude loaded the description of the `commit-message` skill into its context. When the developer typed her request, Claude scanned the descriptions of the available skills and assessed whether any of them matched the request. If the description does not contain language that resembles the developer's request closely enough, the assessment fails, and the skill does not fire. The skill is installed correctly. The body is written correctly. But the description

让我们把失败那一刻"到底发生了什么"走一遍。在 session 启动时,Claude 把 `commit-message` skill 的 description 加载进了上下文。开发者敲下她的请求之后,Claude 把"可用 skill 的 description 们"扫一遍,评估其中是否有哪一条与该请求匹配。如果 description 里没有"在语言上与开发者的请求足够相似"的内容,这次评估就会失败,skill 不触发。Skill 装得对,正文写得对——唯独 description——

← Chapter 5 上一页

后一页 →

> has not done its job, which is to bridge the gap between the developer's natural phrasing and the skill's identity.

没把自己的活干完——那份活,正是"在开发者的自然措辞"与"该 skill 的身份"之间搭一座桥。

> When this happens, the fix is almost always to expand the description. Add the phrasings the developer actually uses. Add synonyms. Add the verbs and nouns from the developer's tone of voice. The description above already includes several alternatives: the canonical "asks for a commit message," and the alternatives "write a commit," "what should I commit this as," and "indicates they have staged changes and want a message written." These multiple framings are not redundant. They are the surface area of the skill, the places where natural language can land and still be recognized as a request for this skill.

一旦发生这种情况,修法几乎都是去"扩充 description":把开发者实际会用的措辞加进去、把同义词加进去、把开发者语气里的动词与名词加进去。本例的 description 实际上已经包含了好几种说法——典型说法"asks for a commit message",以及几条变体"write a commit"、"what should I commit this as",再加上"indicates they have staged changes and want a message written"。这些多角度的措辞,不是冗余——它们就是这份 skill 的"表面积",是自然语言可能落脚、却仍然能被识别为"请求该 skill"的所有落脚点。

> A useful diagnostic exercise, when a skill fails to fire, is to ask Claude directly. Type into the session, `why did the commit-message skill not fire on my last request?` Claude will often produce a useful answer, naming the gap between the request and the description. That answer is then a precise instruction for what to add to the description. After making the change, restart the session for the change to take effect, and try again. Within two or three iterations, almost every triggering problem can be solved by description refinement.

一份好用的诊断手法,是在"skill 不触发"时直接问 Claude。在 session 里敲一句 `why did the commit-message skill not fire on my last request?`,Claude 通常会给出一份有用的回答,把"请求与 description 之间的空缺"直接点出来。那段回答,就相当于一份精确的指令:你该往 description 里加什么。改完,重启 session 让变更生效,再试一次。两到三轮迭代之内,几乎所有"该触发却不触发"的问题,都能通过细化 description 解决。

There are two other failure modes worth naming, though both are less common than the triggering problem. The first is the over-firing problem, in which the description is so broad that the skill activates on requests that should not have triggered it. A `commit-message` skill whose description merely says "anything related to git" will fire when the user asks how to revert a commit, which is the wrong behavior. The fix is to narrow the description by adding negative phrasings or tighter scope language. The second failure mode is the body problem, in which the skill fires correctly but produces output that does not match what the body specifies. This is almost always a sign that the body is ambiguous in some way that the developer did not notice when writing it. The fix is to read the body as if reading it for the first time, find the place where instruction and output diverge, and tighten the language. Examples in the body, especially good and bad examples placed side by side, are extraordinarily effective for this.

还有两种失败模式值得点出名字,虽然都不如"不触发"那么常见。第一种是"过度触发"问题:description 写得太宽,导致 skill 在不该激活的请求上也激活了。一份 `commit-message` skill,如果 description 只写"anything related to git",那用户在问"怎么 revert 一个 commit"时也会触发——这是错误行为。修法是把 description 收窄:加否定性措辞,或者用更紧的 scope 表述。第二种是"正文问题":skill 触发正确,但产出的输出与正文规定不符。这几乎总是一个信号——正文里有某处歧义,是作者写的时候没注意到的。修法是把自己当作"第一次读这份正文"的读者,找出"指令与输出"开始分叉的那个点,把语言收紧。在正文里放示例,尤其是"好例子与坏例子并列",对治这种问题异常有效。

The Skill in Real Use · 把 Skill 用到真实场景

← Chapter 5 上一页

后一页 →

The skill is now built, installed, and verified. The developer has, in roughly twenty minutes of focused work, installed a piece of automation that will pay her back every single time she commits code for the rest of her time using Claude Code. The investment is small. The dividend is daily, indefinite, and silent. There will be no celebration in the terminal when the hundredth commit message gets written by this skill. The developer will simply notice, weeks from now, that her commit history reads more cleanly than it used to, and that she stopped having to think about commit messages somewhere along the way. This is what skills, in aggregate, do to a developer's working life. Each one is small. The compounding is large.

Skill 已经搭好、装上、验证过。开发者用大约二十分钟的专注工作,装进了一份"自动化"——它会在她今后用 Claude Code 的每一次 commit 中,反复回赠给她。投入很小;分红,则是日复一日、绵延不绝、悄无声息的。第一百条 commit message 被这条 skill 写出来的那一刻,终端里不会有任何庆祝;几周之后,这位开发者只是会忽然发现:自己的 commit 历史读起来比从前顺多了,而在某个不知名的时点之后,她再也没为 commit message 操过心。这,就是 skill 们,作为一份"集合",对开发者职业生涯真正做的事——每一条都很小,复利却很大。

A worked debugging walkthrough is worth including here, because the experience of debugging a skill is one of the moments that most reliably separates developers who go on to build serious skill libraries from those who give up after one or two attempts. The walkthrough is from a real session. The skill author had built a skill called `explain-error`, intended to produce structured debugging guidance from a stack trace. She tested it by pasting a Python traceback and asking, `what does this mean`. The skill did not fire. Claude produced a reasonable answer, but not the structured output the skill body specified. The author was confused, because the description seemed clear. She typed into the same session, `why did the explain-error skill not fire on my last request`. Claude responded with a short paragraph: the description mentions errors and stack traces, but the user's request used the word *means* rather than any of the canonical phrasings, and the description had no language that bridged the gap between asking what something means and asking for an error explanation.

这里值得插入一份"完整的 debug 走读",因为"调一个 skill"的体验,正是最能把"会继续搭出严肃 skill 库的人"与"试一两次就放弃的人"区分开来的那种时刻。这份走读来自一次真实的 session。Skill 作者搭了一个叫 `explain-error` 的 skill,本意是"基于一段 stack trace,产出结构化的调试指引"。她用粘贴一段 Python traceback 并问 `what does this mean` 来测试。Skill 没触发;Claude 给了一份合理的回答,但不是 skill 正文规定的那种结构化输出。作者很困惑,因为 description 看起来清清楚楚。她在同一个 session 里敲了一句 `why did the explain-error skill not fire on my last request`。Claude 回了一段简短的诊断:description 里提到了 errors 和 stack traces,但用户请求用的是 *means* 这个词,并不在那些典型措辞之列;description 里没有"在'问某事是什么意思'与'请求一个错误解释'之间搭桥"的措辞。

> This was a precise diagnosis. The author edited the description, adding the phrasing `what does this mean`, `what is going on here`, and `what is happening` to the trigger language. She restarted the session. She pasted the same traceback and asked, `what does this mean`. The skill fired. The output followed the structured format the body specified. The total time from first failure to working skill was roughly four minutes. The lesson the author took away was not about this specific skill. The lesson was about the general method: when a skill fails to fire, ask Claude why, edit the description in light of the answer, restart, and try again. This loop converges quickly, almost always within two or three iterations. Developers who

这是一份精确的诊断。作者随后编辑了 description,加进 `what does this mean`、`what is going on here`、`what is happening` 这几条触发语言;重启 session;再粘贴同一段 traceback,问 `what does this mean`——这次 skill 触发了,输出严格遵守正文中规定的结构化格式。从第一次失败到"能跑",总共花了大约四分钟。这位作者从这件事里带走的,不是关于"这个具体 skill"的什么道理,而是关于*通用方法*的道理:当一个 skill 该触发却不触发时,去问 Claude 为什么;依着答案去改 description;重启,再试。这个回路收敛得很快,几乎总在两到三轮之内完成。那些——

← Chapter 5 上一页

CHAPTER SIX →

> internalize the loop stop being intimidated by description writing, because the feedback cycle is fast enough that experimentation is cheap.

把这条回路真正内化的人,从此不会再被"写 description"这件事吓住——因为反馈周期已经快到让"做实验"变得很便宜。

> A second worked build will reinforce the procedure. The skill being built is called `slack-update`. Its job is to take a description of what the developer did today and produce a short, well-formatted Slack message suitable for posting to a team channel as an end-of-day update. This is a skill that fits a specific context but appears in almost every working developer's life: at four-thirty in the afternoon, the developer wants to summarize what she did today, in a format that fits the team's norms, with the right level of detail for an audience who was not in her head while she did the work.

我们再用第二个完整搭建,把整套流程强化一遍。要搭的 skill 叫 `slack-update`。它的职责是:把"开发者今天都做了啥"的一段描述,变成一条简短、格式良好的 Slack 消息,适合作为"日终更新"贴到团队频道里。这条 skill 适用的场景很具体,却几乎出现在每一位在职开发者的生活里——下午四点半,她想把今天做的事概括一遍,得用合乎团队规范的那种格式、用一个"今天并不在开发者脑子里"的听众能承受的那种粒度。

Reflection on the Build · 搭建复盘

> The architectural thinking, applied to this skill, runs like this. The skill belongs at the user level, because end-of-day updates are personal and follow the developer across projects. The skill needs the ability to read recent commits or recent file changes, because those are the raw material from which the update is composed; that argues for `Bash` and `Read` in `allowed-tools`. The skill does not need write access, because the developer will paste the result into Slack herself. The description should fire on phrasings like `give me a Slack update`, `what should I post in standup`, `write my end-of-day message`, and similar.

把这套"架构思考"套到这条 skill 上,跑出来是这样的:它该放在 user 级,因为"日终更新"是个人的,且跟着这位开发者穿越各个项目;它需要有读"近期 commit"或"近期文件改动"的能力——那是从中萃取出更新内容的原材料——所以 `allowed-tools` 里要带 `Bash` 和 `Read`;它不需要写权限,因为用户会自己把结果粘进 Slack;description 应当能在 `give me a Slack update`、`what should I post in standup`、`write my end-of-day message` 这一类措辞下被触发。

```
---
name: slack-update
description: Produces a short, well-formatted Slack message summarizing what
  the developer accomplished today. Use whenever the user asks for "a Slack
  update," "an end-of-day message," "what should I post in standup," "a
  daily update," "summarize my day," or describes wanting to share progress
  with the team. This skill is for outbound updates the developer is
  writing, not for summarizing other people's messages.
allowed-tools: Bash, Read
---

# Slack Update

When asked for a Slack update, follow this procedure.

## Step 1: Gather material
```

> *[注:代码块原文保留,以下为本页中译补充]*

这份示例与上一份 commit-message 是"同一族兄弟":同样的 user 级 + 同样的 `allowed-tools: Bash, Read` + 同样的"先列步骤再做活"骨架。注意 description 末尾特意加了一句"This skill is for outbound updates the developer is writing, not for summarizing other people's messages"——主动划清"我做什么"与"我不做什么"的边界,正是前文总结过的"主动消除歧义"那一招的活学活用。第 1 步 "## Step 1: Gather material" 会在下一页接续展开。

← Chapter 5 上一页

后一页 →

> The `slack-update` skill, like the `commit-message` skill before it, took roughly ten minutes to write once the architectural decisions had been made. The body is longer because the task has more structure, but the discipline is the same: name the steps, specify the format down to the punctuation, and leave no ambiguity about tone or content. The skill, once installed, will compose end-of-day updates that fit a consistent shape across every working day, and the developer will stop spending the four minutes per day she used to spend on this small but real task.

`slack-update` 这条 skill,和前面的 `commit-message` 一样,等架构决策一旦定下来,真正写也就花了大约十分钟。正文更长,是因为这件事本身的结构更细,但纪律仍然是同一条:把步骤命名清楚、把格式规定到标点级别、语气与内容都不留歧义。这条 skill 装上之后,每天

产出的"日终更新"都会套进同一个形状里;而开发者每天为这件"小但真实"的工作省下的那四分钟,也就此一笔勾销。

> There is a final point worth making about the experience of building skills, which only becomes visible after several have been built. The first

关于"搭 skill"这件事的体验,有一个最后想说的观察,它在搭过若干条 skill 之后才会浮现。头一

```
If the user has described what they did today, use that as the primary
material. If they have not, run `git log --since="this morning"
--author="$(git config user.email)"` to surface today's commits, and
ask the user to confirm or add any uncommitted work.

## Step 2: Categorize

Group the work into at most three categories: shipped, in progress, and
blocked. If a category is empty, omit it.

## Step 3: Format

Produce the message in this format:

*Today*
- [bullet, one short line per shipped item]

*In progress*
- [bullet, one short line per in-progress item, with a note on what's next]

*Blocked*
- [bullet, what's blocked and what would unblock it]

Use Slack markdown: asterisks for bold, no headings, plain bullets. Keep
each bullet under 100 characters. The whole message should fit in a
single screen without scrolling.

## Step 4: Tone

Match the developer's tone. If the work was routine, keep it factual.
If something genuinely went well, allow a single short positive note.
Do not invent enthusiasm or use phrases like "exciting progress" unless
the developer has signalled that mood.

Do not include items the developer did not mention or that do not appear
in the day's commits.
```

> *[注:代码块原文保留,以下为本页中译补充]*

这一段是 `slack-update` skill 正文的剩余部分。第 1 步"Gather material"先尊重"用户主动描述"为主原料,缺则用 `git log --since="this morning"` 去挖今日 commit,并请用户确认或补未提交的工作;第 2 步"Categorize"将工作归入至多三类——shipped / in progress / blocked,哪一类空就省;第 3 步"Format"直接规定三条 Slack markdown 模板(*Today* / *In progress* / *Blocked*),每条 bullet 不超过 100 字符、整条消息一屏装下;第 4 步"Tone"对语气做了相当细的拿捏——常规就写实,有好事容许一句小赞,不许生造热情;末了硬性规定"不许写进用户没提的、或当日 commit 里没有的事项"——这一条,正是"不无中生有"那条职业操守在该 skill 上的具体落地。

skill is intimidating. The second skill is easier. By the fifth skill, the act of building has become almost automatic: the developer thinks of a recurring friction, knows immediately that a skill could absorb it, and writes the skill in fifteen minutes. The barrier to building the next skill, after the first few, is no longer technical. It is a matter of recognizing the friction in the moment and choosing to invest fifteen minutes in eliminating it permanently. Most developers do not invest those fifteen minutes, because they are busy with whatever produced the friction in the first place. The developers who do invest those fifteen minutes accumulate, week by week, an environment that becomes increasingly tailored to the way they actually work, and the gap between their environment and the environment of a colleague who does not invest grows wider over time.

条 skill 让人发怵;第二条,轻松一些;到第五条的时候,"搭 skill"这件事已经几乎变成本能——开发者察觉到一个反复出现的摩擦,立刻就知道"这条摩擦能喂给一条 skill",然后用十五分钟把它写出来。最初几条之后,搭下一条 skill 的门槛,已经不再是技术性的。它变成了一件事:在摩擦出现的当口*认出*它,并选择投入那十五分钟,把它从"永远的摩擦"变成"永远的消失"。大多数开发者,不会投入那十五分钟——他们正忙着应付"摩擦"本身。而那些愿意投入的开发者,会一周一周地累积出一个越来越贴合自己工作方式的环境;他们的环境与"不愿投入"的同事之间的鸿沟,会随时间越拉越宽。

A small habit is worth recommending here. The reader who is just beginning her skill library should keep a running list, in a plain text file, of frictions she notices during her working day. The list does not need to be sorted, prioritized, or organized. It just needs to exist. At the end of each working week, she opens the list and picks one or two items to convert into skills over the weekend. The list keeps her honest about which frictions are real and recurring versus which are rare and one-off. The weekly review keeps the library growing at a sustainable pace, neither too fast nor too slow. After three months of this practice, most developers find that they have a library of fifteen to twenty skills, all of them addressing real frictions, all of them used regularly, and all of them earning their place. This is the steady-state pattern that produces lasting results, and it requires no more than thirty minutes per week of deliberate investment.

这里要推荐一条小习惯。刚开始建 skill 库的读者,可以备一份"摩擦清单",放在一份纯文本文件里,记下她每个工作日里观察到的摩擦。清单不必排序、不必分优先、不必整理——它只需要"存在"。每个工作周结束,她打开清单,挑出一两条,周末把它们各自搭成一条 skill。这份清单让她对自己保持诚实:哪些摩擦是真实且反复出现的,哪些是偶发一次性的。每周一次的复审,让 skill 库以"不快不慢"的节奏持续增长。坚持这种做法三个月之后,大多数开发者会发现自己手上已经有十五到二十条 skill——每一条都对应着真实的摩擦,每一条都被经常使用,每一条都"挣得"自己那一席之地。这,就是能产生持久结果的"稳态模式",而它每周只需*不多于*三十分钟的刻意投入。

The next chapter takes the most important field in any skill, the description, and gives it the attention it deserves. The triggering problem encountered in this chapter has a craft, and the craft can be taught. The reader who masters the craft of writing descriptions will, from then on, build skills that fire reliably on the first try, and will spend her time on the body, where the substance of the skill actually lives. That is the subject ahead.

下一章会挑出任何 skill 里最重要的那个字段——description——把它应得的那份关注补足。本章遇到的"该触发却不触发"问题,是一门手艺;这门手艺是可以教的。一旦读者真正掌握了 description 的写作手艺,从此以后,她搭出的 skill 都会在第一次尝试时就稳稳触发;她可以把时间花到正文上——那才是 skill 真正"重"的地方。这正是前方要展开的课题。

CHAPTER SIX · 第六章
Skill Description Engineering: The Art of Triggering
Skill Description 工程:触发手艺

> *"To name a thing accurately is to make it findable. To name it inaccurately is to lose it forever in plain sight."*
>
> — a librarian, anonymous

"把一样东西叫准了,它就找得到;叫偏了,它就在你眼皮底下永远地消失。"

—— 一位图书馆员,匿名

This is the most important chapter in Part Two. The body of a skill is what Claude does when the skill fires. The description is what determines whether the skill fires at all. A perfect body attached to a poor description is a skill that almost never runs. A modest body attached to an excellent description is a skill that runs reliably across the full range of natural phrasings real users employ. Between the two, the description matters more, because a skill that does not fire might as well not exist. This chapter is dedicated to the craft of description writing, and the goal is that by the end of it, the reader will produce descriptions that fire correctly on the first attempt, every time.

这是第二部分里最重要的一章。Skill 的正文,是"该 skill 触发之后 Claude 做什么";description,是"该 skill 到底会不会触发"。一段完美正文挂在一段蹩脚 description 下面,会变成一条几乎永远跑不起来的 skill;一段朴素正文挂在一段优秀 description 下面,会变成一条在用户千变万化的自然措辞下都稳稳跑起来的 skill。在这两者之间,description 更重要——因为一条不触发的 skill,等同于不存在。本章专门献给 description 写作这门手艺;目标,是你读完之后,每次都能在第一次尝试时就写出"一上来就触发得对"的 description。

Begin with what a description actually does mechanically. At the start of a Claude Code session, the system loads the metadata of every available skill, and the description is the most substantive part of that metadata. When the user submits a prompt, Claude evaluates whether any of the loaded descriptions match the intent of the prompt. The matching is semantic, not literal. Claude is not running a regular expression over the description. It is reading the description in the same way it reads any other natural language, and forming a judgment about whether the prompt fits what the description is talking about. If the judgment is yes, the skill body is loaded into context, and the response is shaped by the body. If the judgment is no, the skill is ignored.

先来看 description 在机制层面到底干了什么。在一个 Claude Code session 启动时,系统会把"所有可用 skill 的元数据"加载进来,而 description 是这份元数据里最有实质内容的部分。用户提交一条 prompt 时,Claude 会去评估:已加载的那些 description 中,有没有哪一条与这条 prompt 的意图相匹配。这种匹配是*语义*的,不是字面意义上的。Claude 不会拿着正则表达式在 description 上跑一遍;它读 description 的方式,与读其他任何自然语言一样,然后做出一个判断:这条 prompt 是不是 description 所谈论的那种事。如果判断为"是",该 skill 的正文被加载进上下文,响应被正文塑形;如果判断为"否",该 skill 就被忽略。

This mechanism has two implications that govern everything in this chapter. The first implication is that the description must be a piece of writing that informs Claude's judgment. It is not a label. It is not a tag. It is a short paragraph that should help Claude decide, accurately, whether a given prompt is the kind of prompt the skill exists to handle. The second

这一机制,带出两条贯穿本章的隐含规则。第一条:description 必须是"一段能影响 Claude 判断力的文字"。它不是 label,不是 tag,而是一段简短的段落——目的是帮 Claude 准确判断:一条给定的 prompt,是不是那种"该 skill 存在正是为了处理"的 prompt。第二

> implication is that the description is read alongside the descriptions of every other installed skill. Skills compete for attention. A description that distinguishes its skill clearly from neighboring skills will fire correctly. A description that overlaps with another skill's territory will produce uncertainty, and uncertainty produces inconsistent firing.

条:description 是与"其他所有已装 skill 的 description"一并被读的。Skill 们彼此*竞争*注意力。一条能把自己的 skill 与"邻接 skill"清晰区分开的 description,会触发得很准;一条与另一条 skill 的领域相重叠的 description,会制造不确定性,而不确定性,会制造"忽而触发、忽而不触发"的不一致表现。

> With those implications in mind, here is the working framework this chapter teaches. A good description has four parts, and they appear in roughly this order. The first part is the canonical statement of what the skill does. The second part is the trigger language: the kinds of requests, phrasings, or contexts that should activate the skill. The third part is the boundary language: what the skill is not for, where its territory ends. The fourth part is the input expectation: what the skill assumes the environment will look like when it runs. Not every description needs every part in equal measure, but every description benefits from at least considering each of the four.

带着这两条隐含规则在脑里,本章要教的工作框架是这样的:一段好的 description 包含四个部分,大致按以下顺序出现。第一部分——典型陈述:这条 skill 做什么;第二部分——触发语言:应当激活该 skill 的那些请求、措辞或情境;第三部分——边界语言:这条 skill 不做什么,它的领域止于何处;第四部分——输入预期:本 skill 跑起来时,对环境状态的假定。并不是每条 description 都要把四部分平摊到位;但每一条 description 都至少应当把这四个部分都各自考量一遍。

> Begin with the canonical statement. This is the one-sentence version of the skill, the version that would appear in a directory listing. It should answer the question, what does this skill do, in twelve to twenty words. Not all of the words. Just enough to make clear, to a reader who has never seen the skill before, what the skill exists to accomplish. The canonical statement of the `commit-message` skill from the previous chapter would be: `generates a conventional commits message from the currently staged changes in a git repository`. That is the skill, named accurately and concisely. Everything that follows refines it.

先讲*典型陈述*。所谓典型陈述,就是"该 skill 的一句话版",也就是"出现在某个目录清单里"的那个版本。它应当在十二到二十个词内答完"这条 skill 做什么"——不是把所有词都用上,只够让一位从没见过这条 skill 的读者,能弄清楚这条 skill 是为达成什么而存在的。上一章 `commit-message` 的典型陈述就是:`generates a conventional commits message from the currently staged changes in a git repository`。这就是那条 skill——命名准确、措辞精炼。后面所有的部分,都是在为它做"精细化"。

Trigger Language · 触发语言

> The trigger language is the part of the description that does the heaviest lifting. It is where the skill author lists the actual phrasings real users employ when they want this skill to run. Trigger language must be derived from observation, not from imagination. The skill author should write trigger language by asking, what would I actually type when I wanted this skill to run, and what would my colleagues type, and what would a developer who has never used this skill before type if she wanted the same outcome. The variety of phrasings real humans employ is broader than any single author intuits. Useful trigger phrasings often include the canonical

触发语言,是 description 里"最扛活"的那一块——是 skill 作者列出"用户实际会敲的那些措辞"的地方。触发语言必须从"观察"里来,不能从"想象"里来。Skill 作者写触发语言时,应当反复问自己:真要触发这条 skill 时,我会怎么敲?我的同事会怎么敲?一位从来没用过这条

skill、但想达到同样结果的开发者,会怎么敲?真实人类措辞的多样性,远超任何单个作者的直觉。真正有用的触发措辞,常常会包含那条典型

← Chapter 6 上一页

后一页 →

> request, several casual paraphrases, the verb form and the noun form of the request, and any common abbreviations.

请求本身,加上几条口语化的转述,加上动词形式与名词形式,再加上任何常用的简写。

> The first description fires reliably. The second description fires only when the user happens to use the exact words `PR description writer`, which almost no one does. The difference between the two is not a difference of quality. It is a difference of trigger surface area. The first description offers many places where natural language can land. The second offers one. A skill with one trigger surface and a perfect body is, in practice, a skill that almost never runs.

第一条 description 触发得稳;第二条只有当用户恰好用上 `PR description writer` 这几个字时才触发——而几乎没人会这样敲。两者之间的差异,不在"质量",而在"触发表面积":第一条 description 给自然语言提供了许多可以"落脚"的地方,第二条只提供一个。一条只有一个触发面、却有着完美正文的 skill,实际上就是一条几乎跑不起来的 skill。

> The boundary language matters when the skill operates near the territory of other skills. If the developer also has a skill called `pr-review`, which reviews other people's pull requests, then the `pr-description` skill needs language that distinguishes it. A line such as `this skill is for writing the description of a PR you are creating, not for reviewing one that someone else wrote`, prevents collision between the two. Without that line, an ambiguous request such as `help me with this PR` could activate either skill, and the developer's experience becomes unreliable.

当一条 skill 跑在"邻近 skill 的领地"附近时,边界语言就特别重要。如果这位开发者还有一条叫 `pr-review` 的 skill——它专门 review 别人的 PR——那 `pr-description` 这条 skill 就需要用一段措辞把自己与对方区分开。例如这样一句:`this skill is for writing the description of a PR you are creating, not for reviewing one that someone else wrote`——它能防止两条 skill"撞车"。少了这句话,像 `help me with this PR` 这种带歧义的请求,就可能激活两者中任意一条,开发者的体感就变得不可靠。

> The input expectation is the part of the description that prevents the skill from firing in contexts where it cannot succeed. A skill that operates on staged git changes, like the `commit-message` skill, does no good when there are no staged changes. The description benefits from a phrase such as `assumes the user has staged changes in a git repository`. This phrase informs Claude's judgment in two ways. First, it makes the skill more likely to fire when the user is clearly in the right context. Second, it makes the skill less likely to fire in contexts where the assumption obviously does not hold, such as a non-git directory or a fresh checkout with no changes.

输入预期,是 description 里"防止 skill 在无法成功的场景下触发"的那一块。像 `commit-message` 这样跑在已暂存 git 改动上的 skill,如果没有暂存改动,跑起来毫无价值。Description 里加一句 `assumes the user has staged changes in a git repository`,会有所得。这句话影响 Claude 的判断,从两个方向:其一,在用户明显处在"对的上下文"时,让 skill 更可能触发;其二,在"前提显然不成立"的场景(非 git 目录、刚 checkout 还没改动的仓库)下,让 skill 更不可能触发。

> There is one further technique worth introducing here, which comes from Anthropic's own documentation on skill authoring. Claude has a tendency to under-trigger skills, that is, to err on the side of not firing them when in doubt. The recommended counter-technique is to make

这里还要再介绍一种手法,它来自 Anthropic 官方关于 skill 编写的文档。Claude 存在一种倾向——*欠触发*(under-trigger):当拿不准时,它倾向于"不去触发"。官方推荐的应对手法,是把

```
description: Drafts a pull request description in the team's standard format.
  Use whenever the user asks for a PR description, pull request description,
  PR body, or asks "write a PR for this branch," "make me a PR
  description," "what should the PR say," or describes wanting to summarize
  a branch's changes for review. Always read the diff before writing.

description: PR description writer.
```

> *[注:代码块原文保留,以下为本页中译补充]*

这里把两条"质量悬殊"的 description 并排放在了一起。第一条铺得很开:典型陈述 + 一长串触发措辞(PR description / pull request description / PR body / write a PR for this branch / make me a PR description / what should the PR say),再加一句触发之后的行为约束(Always read the diff before writing)——这是"高质量 description"的典型;第二条只有六个字:`PR description writer`——既无典型陈述、也无触发措辞、更无边界与输入预期。把这两条并排放,差异之显著,几乎不需要再作解释。

← Chapter 6 上一页

后一页 →

> descriptions slightly assertive. A description that says `use this skill whenever the user mentions X` is more effective than a description that merely describes the topic of X. The verb `use`, applied to the user's phrasing rather than to the topic, gives Claude a clearer signal that the skill is meant to be activated rather than merely consulted. This is a small change with substantial consequences. Adding the phrase `use whenever the user mentions` to a description often raises its trigger reliability noticeably without affecting any other behavior.

description 写得稍微"主动"一些——一条写着 `use this skill whenever the user mentions X` 的 description,比一条仅仅"描述 X 主题"的 description,效果要好。把 `use` 这个动词,作用在"用户的措辞"上,而不是"主题"上,会给 Claude 更清晰的信号:这条 skill 是要被激活的,而不只是被"翻阅"的。这是一个小改动,却带来大后果:在 description 里加上 `use whenever the user mentions` 这一句,通常能明显提高触发的可靠性,而不影响任何其他行为。

> Apply all of this to a worked example. The skill is called `explain-error`, and its job is to take an error message or stack trace and produce a clear, prioritized explanation, including likely cause and recommended next debugging step. A weak description for this skill might read, `helps explain errors`. That description is too short, has no trigger language, no boundary, no input expectation, and no `use whenever` phrasing. It will under-fire constantly. A strong description would read as follows.

把以上所有原则套到一份具体示例上。Skill 叫 `explain-error`——职责是:拿到一条 error message 或 stack trace,产出一份清晰、按优先级排好序的解释,包括"最可能的原因"和"下一步建议的调试动作"。这条 skill 的"弱 description"可能只写 `helps explain errors`——太短、没有触发语言、没有边界、没有输入预期、也没有 `use whenever` 句式,会一直欠触发。一段"强 description"则长这样——

```
description: Explains a given error message or stack trace and proposes the
  most likely cause and a concrete next debugging step. Use whenever the
  user pastes an error, stack trace, traceback, exception, or compiler
  output and asks "what is this," "why is this happening," "how do I
  fix this," or otherwise indicates they want help interpreting an
  error. Do not use this skill for general "my code is broken" requests
  where no error message is present; in those cases, ask the user for
  the error first.
```

A Worked Description · 一份完整示例

> Notice the structure. The first sentence is the canonical statement. The middle of the description lists trigger language, including five concrete phrasings the user might employ. The end carries explicit boundary language: `do not use this skill for`, followed by the specific case where the skill should defer to a clarifying question instead. This kind of description fires correctly across the full natural variety of how developers actually report errors.

留意它的结构。第一句是典型陈述;中间列出触发语言——其中含五条"用户可能用到的具体措辞";尾部则是有显式标记的边界语言:`do not use this skill for`,后接那种"该 skill 应当让位给一次澄清性提问"的特殊情况。这种 description,能在"开发者实际报告错误的全部自然多样性"之下,触发得准。

> Two final practices round out the craft. The first is the trigger test. After writing a description, the skill author should ask herself whether she can describe the skill's task in three meaningfully different ways and have all three of them fire the skill. If yes, the description has good coverage. If no, the description should be expanded with the missing phrasings. This three-way test takes a minute to perform and prevents a category of bug that

最后还有两条实践,把 description 这门手艺收齐。第一条,叫"触发测试"。写完一段 description,skill 作者应当问自己:这件事,我能不能用"在意义上彼此不同"的三种说法把它讲出来,且这三种说法都能让该 skill 触发?如果能,description 覆盖得不错;如果不能,就该把缺失的措辞补进 description。这种"三方测试"花不了一分钟,却能防住一类——

← Chapter 6 上一页

后一页 →

> would otherwise only surface weeks later, in a real session, at a moment when the developer is least equipped to debug it.

bug——这些 bug 若不预防,往往要等到几周后,在某次真实 session 里才暴露;而那时,恰恰是开发者最无力 debug 它的时刻。

> The second practice is the false-trigger test, which is the inverse. After writing a description, the skill author should think of two requests that look superficially similar to the skill's domain but should not fire the skill. If the description, as written, would clearly fire on those requests, the description is too broad and needs boundary language. The `commit-message` skill, for example, should not fire when the user asks how to revert a commit, even though the request mentions commits. A boundary line such as `this skill is for writing new commit messages, not for git history operations`, prevents the false trigger.

第二条,叫"误触发测试",正好是上一条的反面。写完 description,skill 作者应当想出两条"表面上像属于该 skill 领域、但其实不该触发该 skill"的请求。如果按当前 description 写法,这两条会明显触发,那 description 就过宽了,需要补上边界语言。比如 `commit-message`

skill,用户问"怎么 revert 一个 commit"时,就不该触发——哪怕那条请求也提到了 commit。一句边界表述 `this skill is for writing new commit messages, not for git history operations`,就能防住这类误触发。

> Description writing is, in the end, a discipline of imagining the user. The author who writes good descriptions is the author who has thought concretely about how real people will phrase requests in real moments. There is no formula that substitutes for this imagination, but the four-part framework, the `use whenever` phrasing, the trigger test, and the false-trigger test together produce descriptions that fire correctly on the first attempt in the large majority of cases.

说到底,写 description 是一种"想象用户"的纪律。能写出好 description 的作者,就是那种"对真实的人在真实瞬间会怎么措辞"有过具体想象的作者。没有任何公式能替代这种想象;但"四段式框架 + `use whenever` 句式 + 触发测试 + 误触发测试"这一整套组合,在大多数情形下,已经能产出"在第一次尝试时就触发得对"的 description。

> A worked rewrite case study makes the framework concrete. The skill is called `api-doc`, and its job is to write OpenAPI documentation for a given route. The first draft of the description, written by a developer who had not yet read this chapter, looked like this.

一份完整的"重写案例",会让这套框架具体起来。Skill 叫 `api-doc`——职责是:为一个给定的 route 写 OpenAPI 文档。第一稿 description,由一位还没读过本章的开发者写成,长这样:

```
description: Writes API documentation.
```

Iterating on a Failing Description · 改写一段失败的 Description

> This description fired roughly one time in five. Reading it through the four-part framework explains why. The canonical statement is acceptable, but it is the only thing the description contains. There is no trigger language, no boundary, no input expectation. A developer who types `document this endpoint`, `write OpenAPI for this route`, or `generate Swagger for this handler` will find that none of those phrasings clearly match the description, because none of them are the words API documentation. The skill exists, but it does not fire when its time comes.

这段 description 的触发率,大约只有五分之一。用"四段式框架"过一遍,原因就很清楚:典型陈述部分是合格的——但整段 description 也*只有*这一部分。没有触发语言、没有边界、没有输入预期。一位开发者敲 `document this endpoint`、`write OpenAPI for this route`、或 `generate Swagger for this handler` 时,会发现这些措辞与 description 都不算清晰匹配——因为它们都不含"API documentation"这几个字。Skill 在那,但该触发的时候,它不触发。

← Chapter 6 上一页

后一页 →

> A second draft, after the developer noticed the under-firing problem and tried to fix it, looked like this.

第二位开发者注意到欠触发问题、试图修一次之后,第二稿长这样:

```
description: Writes API documentation. Use when the user asks for
```

```
  documentation, OpenAPI, Swagger, or wants to document a route.
```

> *[注:代码块原文保留,以下为本页中译补充]*

这版比起第一稿明显好一些:触发语言加进了最常见的三种措辞(documentation / OpenAPI / Swagger),加上 `use when` 这个动词——告诉 Claude "是要被激活的"而不是仅仅"被翻阅"。这一改之后,触发率明显上升。

> This is meaningfully better. The trigger language now includes three of the most common phrasings, and the verb `use` signals that the skill should be activated rather than merely consulted. Firing rate rose noticeably with this change. But the description still has problems. There is no boundary language, which means it will fire on requests like `document this codebase`, where the right skill might be the `explain-codebase` skill from Chapter Four, not `api-doc`. There is no input expectation, which means it will fire even when the user has not pointed Claude at any specific route. The result is occasional false triggers, where the skill fires on requests it cannot actually serve well.

改到这里,触发语言里多了三条最常见的措辞,`use` 这个动词也把"是要被激活"的信号送了出去——触发率肉眼可见地提升。但 description 仍然有问题:它没有边界语言,所以像 `document this codebase` 这种请求也会触发——而那种请求,其实该由第四章那份 `explain-codebase` skill 来接,而不是 `api-doc`。它也没有输入预期,所以即使用户没把任何具体 route 指给 Claude,它也会触发。结果就是"零星的误触发"——该 skill 跑在它其实根本服务不好的请求上。

> The third draft, after the four-part framework was applied in full, looked like this.

把"四段式框架"完整地套上之后,第三稿长这样:

```
description: Writes OpenAPI documentation for a single API route or handler.
  Use whenever the user asks to "document this endpoint," "document this
  route," "write OpenAPI for this," "add Swagger for this," "generate
  API docs for this handler," or otherwise indicates they want OpenAPI
  documentation produced for a specific route. This skill is for
  documenting a single route at a time, not for documenting an entire
  codebase. Assumes the user has identified the route or handler to
  document, either by pasting it or by referring to a specific file.
```

> This description fires reliably and false-triggers rarely. The canonical statement is precise: a single API route, not a codebase. The trigger language enumerates five concrete phrasings the user might employ. The boundary language makes the single-route scope explicit and prevents collision with broader documentation skills. The input expectation establishes that the user has identified what to document, which prevents the skill from firing in contexts where it has nothing to work with. The total length is roughly seventy words, which is on the longer side for a description but well within the range that descriptions can usefully occupy. The trade between length and reliability falls clearly in favor of length here. A description three times longer than necessary that fires correctly is much better than a description one third the length that fires inconsistently.

这一稿,触发得稳、误触发得少。典型陈述很精确——"单个 API route",不是"整个 codebase";触发语言枚举了五条用户可能用的具体措辞;边界语言把"只接单 route"的范围讲明白,防止它与更广义的文档 skill 撞车;输入预期确立了"用户已经指明要文档化什么"——这能防止该 skill 跑在"完全没有原料"的场景里。整段长度约七十个词,对 description 而言算偏长,但仍稳稳落在 description 实际能发挥作用的合理区间内。在"长度"与"可靠性"之间做权衡,本章的态度是相当明确地倾向"长度":一段比必要长度长三倍、却触发得对的 description,远好于一段长度只有它三分之一、却触发得不稳定的 description。

The case study illustrates a general truth. Description quality is a continuous variable, not a binary. A first-draft description usually works some of the time. Each subsequent revision narrows the gap between when the skill should fire and when it does fire. After two or three revisions, the gap is small enough that the skill becomes reliable in daily use. The reader who is patient with this iteration will, over the course of building her first ten skills, internalize the four-part framework so deeply that her first drafts begin to look like other people's third drafts.

这份案例,讲的是一条普适的真理。Description 的质量是一个连续变量,不是二元判断。第一稿 description 通常只在部分时候管用;每一次改稿,都在把"该 skill 应该触发的时刻"与"它实际触发的时刻"之间的差距收窄一截。改上两三轮之后,差距已经小到让该 skill 在日常使用中变得可靠;而一个愿意耐心走完这几轮迭代的读者,在搭出前十条 skill 的过程中,会把"四段式框架"内化到那种程度——自己写出的第一稿,看起来已经是别人改到第三稿的样子。

A Rubric for Description Quality · 一份"Description 质量"的评分表

A small rubric is worth keeping in working memory while writing descriptions. A description scores well when it answers four questions clearly. What does this skill do, in one sentence. What kinds of phrasings should fire it. Where does this skill stop and another skill begin. What does this skill assume about the input. A description that does not answer any one of these four questions has a corresponding weakness, and the weakness will manifest in a corresponding failure mode. Under-firing usually traces back to weak trigger language. Over-firing usually traces back to weak boundary language. Firing in the wrong context usually traces back to weak input expectations. Firing on the wrong topic entirely usually traces back to a weak canonical statement.

写 description 时,心里值得存一份小评分表。一段 description 得分高,意味着它清晰地答出了四个问题:这条 skill 用一句话讲做什么?哪些措辞应当触发它?这条 skill 在哪里收手、把舞台让给另一条 skill?这条 skill 对输入做了哪些假定?任何一个问题没答上,都会在 description 上留一处对应的弱点;那一处弱点又会表现成某一种对应的失败模式:欠触发通常源于触发语言太弱;过触发通常源于边界语言太弱;在错误的上下文里触发,通常源于输入预期太弱;整段完全跑偏,通常源于典型陈述太弱。

One additional practice is worth introducing here, although its full development belongs in a more advanced treatment of skill design. Anthropic's own `skill-creator` skill includes an evaluation procedure in which the skill author writes twenty test queries, mixing prompts that should fire the skill with prompts that should not, and checks whether the description correctly distinguishes between them. The procedure is straightforward to perform manually. After writing a description, the author lists ten phrasings that should clearly fire the skill and ten that should clearly not. She reads each one and asks, would my description, as written, lead Claude to fire the skill on this query. If the answer disagrees with the intended firing in more than one or two cases out of twenty, the description needs work. If it agrees in all twenty cases, the description is probably ready.

这里还要再介绍一条实践——虽然它的完整展开属于更进阶的 skill 设计话题。Anthropic 自己的 `skill-creator` skill 包含一套评估流程:skill 作者写出二十条测试 query,其中混进"应当触发"与"不应触发"两种,再核验 description 能否正确区分两者。这套流程,人工手动跑也很直白:写完 description 之后,作者列十条"应当明显触发"的措辞,再列十条"应当明显不触发"的措辞;对每一条都问自己:"按当前 description 的写法,这条 query 会让 Claude 触发这条 skill 吗?"如果二十条里有超过一两条的答案与"本意"相左,description 还需再改;如果二十条都答对,那 description 差不多可以交付了。

This systematic check transforms description writing from a craft based on intuition into a craft based on evidence. The intuition is still valuable; it

这种系统性检查,把 description 写作从"凭直觉"的手艺,变成了"凭证据"的手艺。直觉仍然宝贵,它——

← Chapter 6 上一页

后一页 →

> suggests the initial language, the candidate phrasings, the likely failure modes. But the check ensures that the intuition has produced the right result. The author who runs the twenty-query check on every description she writes builds, over time, a sense of which kinds of phrasings tend to be dangerous, which kinds of boundaries tend to be necessary, and which kinds of input expectations tend to matter. The sense becomes intuitive, and her first-draft descriptions begin to pass the twenty-query check on the first attempt. This is the path to becoming a skilled skill author. There is no shortcut, but the path is short, and the reader who walks it deliberately reaches the destination within a working week.

提供最初的措辞、候选的触发语句、以及可能的失败模式;而这套"测试"则保证,直觉真的产出了正确的结果。每条 description 都跑一遍"二十条 query"检查的作者,会随着时间逐渐培养出一种"对哪类措辞容易出事、哪类边界常常必要、哪类输入预期通常关键"的判断力;这种判断力会内化成本能,她写出的初稿 description 开始在第一次尝试时就通过那二十条 query 的检查。这,就是"成为熟练 skill 作者"那条路。没有捷径,但路不长——愿意有意识地走下去的读者,在一个工作周之内就能走到终点。

> Description writing pays out at every layer of the skill library. Better descriptions mean more reliable firing, which means the developer trusts the library, which means she invests in extending it, which means the library grows, which means the compounding accelerates. The single largest difference between a developer with a thriving skill library and a developer with a stalled one is, almost always, description quality. The body of the skill is the visible substance, but the description is the invisible gate that determines whether the body ever runs. Master the gate, and the rest follows.

Description 写作的回报,在 skill 库的每一层都会兑现。更好的 description → 更可靠的触发 → 开发者更信任这个库 → 她更愿意投入去扩展它 → 库继续生长 → 复利进一步加速。一位开发者手上"茁壮成长"的 skill 库,与另一位"原地踏步"的 skill 库之间,最大的单项差异,几乎总是 description 的质量。Skill 的正文,看得到的实质;description,看不见的闸——决定正文到底会不会被跑起来。掌握了这道闸,其余的事会自然跟上。

> One last observation is worth keeping in mind. Descriptions, like any other piece of writing, age. The phrasings a developer uses today are not necessarily the phrasings she uses in six months, because vocabulary drifts as projects change and as new tools enter daily use. A description that fired reliably six months ago may begin under-firing now, not because anything in the skill changed but because the developer has started phrasing her requests differently. The remedy is the same in both cases: add the new phrasings to the description. Treat descriptions as living documents, revisited whenever a skill seems to be firing less reliably than it used to. The cost of revising is small. The benefit is that the library stays in tune with the developer who depends on it.

还有最后一条观察,值得装在脑子里:description,和任何一篇文字一样,会"老化"。开发者今天用的措辞,不一定是她六个月后还在用的措辞——因为词汇会随项目变化、随新工具进入日常而漂移。半年前触发得稳的 description,现在开始欠触发——并不是因为 skill 里有什么变了,而是因为开发者敲请求的措辞在变。修法在两种情况下都一样:把新措辞加进 description。把 description 当作"活文档"——只要某条 skill 看起来"没以前那么稳",就回去看看它。修订的成本很小;收益,是让整个 skill 库始终与它所服务的那位开发者保持同调。

> With description engineering in hand, the rest of the work of building skills can focus on the body, where the substance of the skill lives. That work expands in the next chapter, which moves beyond the single-file skill into the world of bundled resources, scripts, and references that turn a skill from a paragraph of instructions into a substantial piece of installed capability.

把 description 工程握在手里之后,搭 skill 其余的工作就可以把注意力放到正文上——那才是 skill 真正的实质所在。这份工作在下一章会进一步展开:它将跨出"单文件 skill"的范畴,进入"打包资源、脚本、引用"的世界,把一条 skill 从"一段指令文字",变成"一件装在环境里的、像样的能力"。

← Chapter 6 上一页

CHAPTER SEVEN →

CHAPTER SEVEN · 第七章
Multi-File Skills and Bundled Resources
多文件 Skill 与打包资源

> *"A small thing well-organized is a tool. A large thing poorly organized is an obstacle. The line between them is layout."*
>
> — Edwin Bartholomew, woodworker

"一件小东西,组织得好,就是工具;一件大东西,组织得差,就是障碍。两者之间的界线,是'布局'。"

—— 埃德温·巴塞洛缪,木匠

The skills built so far have been single-file skills. `SKILL.md` is the only file in the skill folder, and the entire skill, frontmatter and instructions and any examples, lives within that one document. This works perfectly for skills whose total content is small. For many skills, single-file is exactly the right structure, and adding more files would only complicate something that does not need complication.

到目前为止搭出的 skill,都是"单文件" skill——`SKILL.md` 是 skill 文件夹里*唯*一的文件,整条 skill——frontmatter、指令、示例——全部塞在这一份文档里。对那些"总体内容小"的 skill 而言,这种做法堪称完美;事实上,对很多 skill 来说,单文件*恰恰*是合适的结构,加文件只会徒增不必要的复杂度。

But many of the most valuable skills cannot reasonably fit in a single file. A skill that scaffolds a new React component might need a dozen template files: one for the component itself, one for the test file, one for the Storybook story, one for the CSS module, one for the index file, one for each variant of the component the skill supports. A skill that runs a deterministic data extraction over a PDF might need to bundle the Python script that does the extraction, because asking the language model to do that work would be slow, expensive, and unreliable compared to running real code. A skill that encodes the conventions of a complex API might need a fifty-page reference document that the language model can consult when needed but should not load by default. None of these skills can be built well with a single file. All of them benefit from the multi-file structure that is the subject of this chapter.

但许多最有价值的 skill,实际上无法合理地塞进单文件。一个为新 React 组件搭脚手架的 skill,可能需要一打模板文件:组件本身一份、测试一份、Storybook story 一份、CSS module 一份、index 一份、每一种组件变体各一份。一个要在 PDF 上跑"确定性数据抽取"的 skill,可能需要把那段做抽取的 Python 脚本打包进来——因为让语言模型去做这件事,会比真正跑代码慢、贵、且不可靠。一个把某个复杂 API 规约编码起来的 skill,可能需要一份五十页的参考文档,让语言模型在需要时去查,而不是默认就加载。这些 skill,都不适合用单文件搭;它们都受益于本章的主题——多文件结构。

> The convention for multi-file skills, established by Anthropic and adopted across the open-source skills ecosystem, is straightforward. A skill folder may contain three special subfolders, each with a specific role. The `scripts` subfolder contains executable code. The `references` subfolder contains additional markdown documents that the `SKILL.md` body can ask Claude to read on demand. The `assets` subfolder contains files that are used as templates, fixtures, or other inputs during the skill's execution. Any of the three subfolders may be present or absent in any given skill. Some skills

多文件 skill 的这套惯例,由 Anthropic 确立,并被整个开源 skill 生态所采纳,本身很直白。一个 skill 文件夹可以包含三个特殊的子目录,每个各司其职。`scripts` 目录装可执行代码;`references` 目录装若干额外的 markdown 文档——`SKILL.md` 的正文可以指示 Claude 在需要时去读它们;`assets` 目录装的是"在该 skill 执行过程中作为模板、夹具、或其他输入"的文件。任一个 skill 里,这三个目录*都可以*有、也都可以没有;有些 skill

← Chapter 6 结尾

后一页 →

> use all three. Most use one or two. The folder structure for a skill that uses all three would look like this.

三者都用上;大多数只用到一两个。一个三类都用上的 skill,文件夹结构大致是这样:

```
react-component/
├── SKILL.md
├── scripts/
│   └── generate.sh
├── references/
│   ├── component-conventions.md
│   └── testing-conventions.md
└── assets/
    ├── component.template.tsx
    ├── test.template.tsx
    └── story.template.tsx
```

> *[注:代码块原文保留,以下为本页中译补充]*

这是一份典型的"全三类"多文件 skill 结构:`SKILL.md` 在最外层做"调度",`scripts/` 装可执行脚本(`generate.sh`),`references/` 装"按需阅读"的两份 markdown(`component-conventions.md` 和 `testing-conventions.md`),`assets/` 装三份模板文件——组件、测试、Storybook 各一份。这种"外层调度 + 三类支撑材料"的结构,正是从这一章起所有"复杂 skill"的统一骨架。

> The naming of the three folders matters because it makes the skill folder predictable to read. A developer encountering the skill for the first time can look at the folder layout and form an immediate, accurate guess about what each piece does. Scripts are for running. References are for reading. Assets are for using as material in the skill's output. This predictability compounds across a skill library; once a developer has internalized the convention, every skill she encounters reveals its structure at a glance.

这三个目录的命名之所以重要,是因为它让"读 skill 文件夹"这件事变得"可预期"。一位第一次碰上这条 skill 的开发者,看一眼目录布局,就能立刻、准确地猜出每一块在做什么:`scripts` 装的是"用来跑的东西";`references` 装的是"用来读的东西";`assets` 装的是"在 skill

输出里用作原料"的东西。这种可预期性会在整个 skill 库里复利——一旦某位开发者把这条约定内化,她以后遇到每一条 skill,都能一眼看清它的结构。

> Begin with `references`, which is the most common form of bundled material. A reference is a markdown file that lives inside the skill folder and contains content that the skill body wants to be available, but that does not need to be loaded into context unless it is actually needed. The classic example is a long format specification. If a skill produces output in a complex format, the format specification might be ten pages. Putting the entire specification in `SKILL.md` would mean that every time the skill activates, ten pages of spec land in the context window, even if only a small section is relevant to the current task. The better pattern is to keep `SKILL.md` 短, place the full specification in a reference file, and have `SKILL.md` instruct Claude to read the reference when relevant.

先讲 `references` ——它是打包材料里最常见的一种形式。所谓 reference,就是一份住在 skill 文件夹里的 markdown 文件,装着"该 skill 正文希望它可用、但只有在真的需要时才加载"的内容。最经典的例子是一份很长的格式规范。如果某条 skill 输出的内容是某种复杂格式,那份格式规范可能有十页;把它整段塞进 `SKILL.md`,意味着该 skill 每次激活时,十页规范都会进上下文窗口——而当前任务真正用到的可能只是其中很小一段。更优的做法是:把 `SKILL.md` 保持简短,把完整规范放进一份 reference 文件,然后让 `SKILL.md` 在相关时指示 Claude 去读那份 reference。

How Loading Works · 加载机制

> The mechanics are simple. The skill body refers to the reference file by relative path, often within an instruction line. Claude, on encountering that instruction, knows that it can read the reference file using its standard file-

机制本身很简单。Skill 正文通过相对路径引用 reference 文件——通常写在某条指令里。Claude 遇到这条指令,就知道它可以用自己的标准文件——

← Chapter 7 上一页

后一页 →

> reading tools. Here is what the relevant portion of a `SKILL.md` file might look like, illustrating reference usage in practice.

读工具,去把那份 reference 读进来。下面是一份 `SKILL.md` 的相关段落,用来演示 reference 在实战中是怎么被用的。

```
---
name: api-doc
description: Writes OpenAPI documentation for a given route. Use whenever
  the user asks to "document this endpoint," "write OpenAPI for this,"
  "add Swagger docs," or otherwise indicates they want API documentation
  generated.
---

# API Documentation

When asked to document an API endpoint, follow this procedure.

1. Read the route handler the user is referring to.
2. If you are unsure about the project's documentation conventions, read
   `references/openapi-conventions.md` for the full style guide.
```

```
3. Read `references/error-format.md` only if the endpoint can return errors
   that are not the standard 4xx or 5xx codes.
4. Produce the OpenAPI fragment in the format described in the conventions
   file.

Do not load the references unless the situation calls for them. The
conventions file is long, and loading it every time wastes context.
```

[注:代码块原文保留,以下为本页中译补充]

这份 `api-doc` 的 `SKILL.md` 是一份绝佳的"reference 触发模式"示范。注意正文刻意很短,只列了四步流程的高层骨架;真正的"风格指南"放在 `references/openapi-conventions.md`,"错误格式"放在 `references/error-format.md`。第 2 步用 `If you are unsure ... read X` 模式——这是"按需加载 reference"的标准指令形式;第 3 步的 `only if ...` 是更严格的"条件性加载",用一句话把"何时读"钉死;最后那句"do not load the references unless the situation calls for them"是元指令,提醒 Claude "别养成每次都加载"的习惯——这正是上一章反复强调的"避免浪费 context window"那条纪律在 multi-file 场景下的具体化。

Several details in that example are worth highlighting, because they reflect the discipline of good multi-file design. First, the body is short. The body's job is to describe the procedure at the highest level. The detail lives in the references, where it does not cost context unless needed. Second, the body explicitly tells Claude when to load each reference. The instruction `Read X if Y` is the language pattern that triggers progressive disclosure correctly. Without that instruction, Claude might either always load the references, defeating their purpose, or never load them, leaving important information unused. Third, the body is honest about the cost of loading references. The closing line, `do not load the references unless the situation calls for them`, is a small but real piece of meta-instruction that helps Claude make the right call.

这份示例里有几处细节值得拎出来,因为它们正是"好的多文件设计"纪律的体现。第一,正文很短。正文的职责,只到"在最高层描述流程"为止;细节放在 reference 里——除非真用上,否则不占 context。第二,正文显式地告诉 Claude 何时加载每份 reference。`Read X if Y` 这种指令句式,正是"正确触发渐进式披露"的语言模式。少了它,Claude 可能会"每次都把 reference 加载进来"——这样 reference 就没意义了;也可能会"一次都不加载"——这样 reference 里的重要信息就被白白闲置。第三,正文对"加载 reference 的代价"是坦白的。末了那句 `do not load the references unless the situation calls for them`,是一段不起眼但确有分量的元指令——它帮 Claude 在该加载时加载,在不该加载时不加载。

Now turn to scripts. Scripts are executable code bundled inside a skill folder, intended to be run by Claude as part of carrying out the skill. Scripts solve a category of problem that prose instructions handle poorly: deterministic, repeatable operations that real code can perform faster, cheaper, and more reliably than a language model can simulate. Sorting a

现在把目光转向 script。所谓 script,是打包在 skill 文件夹里、供 Claude 在执行该 skill 时调用的可执行代码。Script 解决的是"光靠散文指令处理不好"的那一类问题:确定性的、可重复的操作——真代码来做,比语言模型去"模拟"更便宜、更快、更可靠。给一个数组排

← Chapter 7 上一页

后一页 →

list, parsing a binary file, querying a database, calling an external API with a known schema. All of these are operations that should be code, not prose instructions to a model. The skill author writes the script once, places it in the `scripts` folder, and the skill body instructs Claude to run it at the appropriate moment.

序、解析一个二进制文件、查一次数据库、用一个已知 schema 去调外部 API——这些都该是*代码*的工作,而不该是对模型的"散文指令"。Skill 作者把脚本写一遍,放进 `scripts` 目录,skill 正文在合适时点指示 Claude 去跑它。

> A useful pattern is to combine a thin script with prose instructions about how to interpret its output. The script does the deterministic work. Claude reads the script's output and applies judgment to it. The combination is faster and more reliable than either approach alone. Anthropic's own document-handling skills follow this pattern. The `pdf` skill, for example, bundles a Python script that extracts form fields from a PDF without loading the PDF into the language model's context. The body of the skill then describes how to use the extracted data to fill or analyze the form. The script handles the part that requires precision. The body handles the part that requires understanding.

一种有用的模式是:把一段"薄脚本"与"关于如何解读其输出的散文指令"组合起来——脚本做那部分确定性的工作;Claude 读完脚本的输出,再把判断力加上去。这种组合,比单用任何一种都快、且更可靠。Anthropic 自家的文档处理 skill 就是这种模式。比如 `pdf` 这条 skill,打包了一段 Python 脚本,用来从 PDF 里抽 form 字段,却不让整份 PDF 进语言模型上下文;skill 正文则描述怎么用抽出来的数据去填表或做分析。脚本负责"需要精度"的那部分,正文负责"需要理解"的那部分。

> Assets, the third bundled resource type, are files used as material in the skill's output. Templates are the dominant case. A skill that scaffolds a new file from a template can keep the template as a separate file in the `assets` folder, and the body of the skill can instruct Claude to copy the template, fill in the variable parts, and write the result to the appropriate location in the project. This pattern is common in code-generation skills, document-generation skills, and any skill where the structure of the output is large and stable, but the specific values change with every invocation.

Asset,第三类打包资源,指的是"在 skill 输出里被当作材料"的文件。模板(是 asset 最常见的一种形态。一条"按模板搭一个新文件"的 skill,可以把模板作为单独文件放在 `assets` 目录里;skill 正文则指示 Claude 把模板拷出来、填好变量位、再把结果写到项目里合适的位置。这种模式在"代码生成"型 skill、"文档生成"型 skill,以及任何"输出结构大而稳定、但具体数值每次都变"的 skill 里,都很常见。

A Worked Multi-File Skill · 一份完整的多文件 Skill 示例

> A worked example brings the three resource types together. Consider a skill called `scaffold-feature`, intended to generate the scaffolding for a new feature in a TypeScript codebase. The scaffolding includes a route file, a service file, a repository file, a test file, and an OpenAPI fragment. The skill folder might contain templates for each of those file types in `assets`, a small Python script in `scripts` that runs the test scaffolding through a formatter, and a reference file in `references` that explains the codebase's naming conventions. The `SKILL.md` body would describe the procedure: ask the user for the feature name, read the conventions reference, copy each template from `assets` while substituting the feature name, run the formatter script over the result, and write the files into their proper locations. Each of the

一份完整的示例把三类资源并到一起。考虑一条叫 `scaffold-feature` 的 skill——它的职责,是为一个 TypeScript 代码库里的新功能生成脚手架。脚手架包含:route 文件、service 文件、repository 文件、test 文件,以及一段 OpenAPI 片段。这个 skill 的文件夹里:`assets` 装每种文件类型的模板;`scripts` 装一段小 Python 脚本,用来把脚手架里的 test 部分过一遍 formatter;`references` 装一份文件,讲清代码库的命名规约。`SKILL.md` 的正文描述整个流程:向用户问新功能的名字;读 conventions reference;从 `assets` 拷出每份模板,边拷边把功能名替换进去;把结果跑一遍 formatter 脚本;再把文件写到项目里各自该去的位置。三类资源中的每一类——

← Chapter 7 上一页

后一页 →

> three resource types is doing what it does best, and the combined skill produces a substantial scaffolding operation in seconds.

——都在做它最擅长的事;合在一起的这条 skill,能在几秒内产出一份像样的脚手架操作。

> A practical limit is worth stating. `SKILL.md` should stay under approximately five hundred lines, even with bundled resources available. The body is read every time the skill fires, and a body that grows past five hundred lines starts to consume noticeable context on every activation. When a body approaches that limit, the right move is to factor surplus content into reference files. The body becomes a router that points Claude to the right reference at the right moment. This is, again, the progressive disclosure principle, applied recursively. The skill keeps its small footprint when it is not needed. It expands gradually as the work demands.

有一条实用的"上限"值得明说:即便已经用上打包资源,`SKILL.md` 也应控制在五百行左右。Skill 每次触发都要读正文;一旦正文超过五百行,每次激活都会开始吃掉可见的 context。当正文逼近这条上限时,正确的做法是把多出来的内容拆成 reference 文件;正文则退化为一份"路由器"——在合适的时刻,把 Claude 指向正确的 reference。这正是"渐进式披露"原则的递归应用:该 skill 在不需要时,保持轻盈的小脚印;在任务需要时,才一点点展开。

> Token budgeting matters at a skill-library level too. A library of forty skills, each with a thirty-line description, a hundred-line body, and several hundred lines of references, might amount to ten thousand or more lines of total content. Loaded all at once, that would dominate any reasonable context window. Loaded progressively, with only metadata at startup and only the relevant body and references on activation, the total cost on any given turn is small. The progressive disclosure architecture is what allows skill libraries to scale without becoming a tax on every prompt. The discipline of building skills that respect this architecture is, in effect, the discipline of writing skills that play well with their neighbors.

"Token 预算"在 skill 库这一层级同样要紧。一个有四十条 skill 的库,每条有三十行的 description、一百行的正文、几百行的 reference,合起来可能是一万行以上的内容——若一次性全部加载,会占满任何一个合理大小的 context window。若按"渐进式披露"加载——启动时只装元数据,激活时只装对应正文与 reference——那么任一特定轮次上,所付的总成本都是小的。"渐进式披露"这套架构,正是 skill 库能够不断扩张、却不对每一条 prompt 都收"税"的原因。把"尊重这套架构"作为搭 skill 的纪律,实际上就是"让 skill 们彼此相处融洽"的纪律。

> A complete worked example of a multi-file skill will make the structure tangible. The skill being built here is called `scaffold-react`. Its job is to scaffold a new React component complete with its test file, its Storybook story, and an index export. The skill is exactly the kind that benefits from bundled resources. The templates for each generated file are stable and substantial, the conventions for component organization are project-specific and worth documenting, and the body itself can stay short by referring outward to the supporting files.

一份完整的多文件 skill 示例,会让这套结构变得"摸得到"。要搭的这条 skill 叫 `scaffold-react` ——职责是:为一个新 React 组件搭好脚手架,包括它的 test 文件、Storybook story,以及一份 index 导出。这条 skill,正是"受益于打包资源"的那种典型:每份生成文件的模板都是稳定且体量不小的;组件组织的规约是项目专属的,值得单独写下来;正文本身则可以保持简短,只是把"指向"打到外面那些支撑文件上。

> The folder structure for the skill looks like this.

这条 skill 的文件夹结构,长这样:

```
scaffold-react/
├── SKILL.md
├── references/
│   └── component-conventions.md
```

```
└── assets/
    ├── component.template.tsx
```

> *[注:代码块原文保留,以下为本页中译补充]*

这是一份"截到本页底部"为止的目录结构展示——`SKILL.md` 在根,`references/` 装组件组织规约,`assets/` 开始装第一份模板。完整的结构会在下一页接续展开,届时 `assets/` 里还会再加入 `test.template.tsx`、`story.template.tsx`、`index.template.ts` 等若干模板文件。

← Chapter 7 上一页

后一页 →


```
├── test.template.tsx
├── story.template.tsx
└── index.template.ts
```

> *[注:代码块原文保留,以下为本页中译补充]*

这是上一页那份目录树在 `assets/` 下的剩余部分——三份模板文件:`test.template.tsx`(测试模板)、`story.template.tsx`(Storybook story 模板)、`index.template.ts`(index 导出模板)。连同上一页的 `component.template.tsx`,这条 skill 一共要"按模板生成"四份文件。

Inside the Skill · 走进 Skill 内部

> The `SKILL.md` file is the entry point. It is short, because the heavy material lives in the `references` and `assets` folders. Its job is to coordinate: to fire on the right requests, gather the input it needs from the user, consult the conventions reference when relevant, and copy the appropriate templates with substitutions. Here is what the body looks like.

`SKILL.md` 文件是入口点。它本身很短——因为"重的素材"都住在 `references` 和 `assets` 里。它的职责是*协调*:在正确的请求上触发、从用户那里收集所需的输入、在相关时去查 conventions reference、把对应模板拷出来做替换。整份正文长这样:

```
---
name: scaffold-react
description: Scaffolds a new React component, complete with test file,
  Storybook story, and index export, in this codebase's conventional
  layout. Use whenever the user asks to "scaffold a component," "create
  a new component," "set up a React component," "make a new component
  called X," or describes wanting the standard files for a new component
  to be created.
allowed-tools: Read, Write, Glob
---

# Scaffold React Component

When asked to scaffold a component, follow this procedure.
```

```
## Step 1: Get the component name

If the user has stated a name, use it. Convert to PascalCase if needed.
If the user has not stated a name, ask once for the name and the
directory it should live in. Default directory is `src/components/`.

## Step 2: Read the conventions

Read `references/component-conventions.md` to confirm the current
conventions for this codebase, including naming, prop typing, and CSS
handling. Apply any conventions that are relevant to the templates.

## Step 3: Generate the files

For each of the four template files in `assets/`, copy it to the target
directory and substitute the placeholder `__COMPONENT_NAME__` with the
PascalCase name.

The four files produced are:
```

> *[注:代码块原文保留,以下为本页中译补充]*

这份 `scaffold-react` 的 `SKILL.md` 把"短正文 + 外部资源"的纪律演示得相当典型:`description` 列举了 scaffold a component / create a new component / set up a React component / make a new component called X 等多条触发措辞;`allowed-tools: Read, Write, Glob` ——只读、写、模式匹配,把其他工具挡在门外;正文"## Step 1"先向用户拿组件名并转 PascalCase,"## Step 2"再按需读 `references/component-conventions.md`(注意"按需"二字,不是每次都加载),"## Step 3"才进入"拷模板、替换占位符"的核心动作——这一段会在下一页接续展开。"生成的四个文件"那条说明,会在下一页详细列出。

← Chapter 7 上一页

后一页 →

> The `SKILL.md` is roughly forty lines. By itself, that is short. But the skill is not actually small. The total content of the skill, including templates and conventions reference, might run two or three hundred lines. The body is short because the body is doing what bodies do best, which is coordination. The substance lives in the supporting files.

`SKILL.md` 自身大约四十行,单看很短。但这条 skill 并不小——连同模板和 conventions reference 在内,整条 skill 的总内容可能要两三百行。正文之所以短,是因为正文在做正文最擅长的事——*协调*。真正的实质,住在那些支撑文件里。

> The `component-conventions.md` reference might look like this.

那份 `component-conventions.md` reference,大致长这样:

```
- `/index.ts` (from index.template.ts)
- `/.tsx` (from component.template.tsx)
- `/.test.tsx` (from test.template.tsx)
- `/.stories.tsx` (from story.template.tsx)
```

```
## Step 4: Confirm

After writing the files, list the four files created and offer to open the
component file for editing.

Do not modify the templates themselves. Substitution happens at copy time.
If the conventions in references/component-conventions.md conflict with
what the templates produce, mention the conflict to the user and ask
which to follow.
```

> *[注:代码块原文保留,以下为本页中译补充]*

这段是上一页那份 `SKILL.md` 的剩余部分——把"四个被生成的文件"逐条列清(每个文件都注明"由哪份模板拷出"),再加"## Step 4: Confirm"——写完之后,列出已建的四个文件,并主动问用户是否要打开组件文件继续编辑。最后那段"Do not modify the templates themselves"是一条很关键的纪律:模板只读,替换发生在"拷出"那一刻;若 `references/component-conventions.md` 与模板产生冲突,要先把冲突亮给用户,再问走哪条——把"判断权"留给人。

```
# React Component Conventions

This codebase follows these conventions for React components.

## Naming

- Component files and folders are PascalCase.
- Hook files start with `use` and are camelCase.
- Test files match the component file with a `.test.tsx` suffix.

## Props

- Define props as a named interface above the component, e.g.
  `interface ButtonProps`.
- Use destructured props in the function signature.
- Default values go in the destructuring, not in defaultProps.

## Styling

- Use CSS modules. The CSS module file matches the component name with
  a `.module.css` suffix.
- Avoid inline styles except for dynamic values.
- Class names are camelCase to match the CSS module convention.

## Exports
```

> *[注:代码块原文保留,以下为本页中译补充]*

这是 `references/component-conventions.md` 的"展示到本页底部为止"的内容:① "## Naming"——组件文件与目录 PascalCase,Hook 文件以 use 起头且 camelCase,测试文件用 `.test.tsx` 后缀;② "## Props"——把 props 定义为命名 interface 放在组件上方、函数签名里用解构、默认值放在解构里而不是 defaultProps;③ "## Styling"——CSS modules、模块文件名 `.module.css` 后缀、避免 inline 样式(动态值除外)、class 名 camelCase 配合 CSS module 约定。"## Exports"那一节会在下一页接续展开。

> The conventions reference is fifty lines. Loaded into context, it costs roughly seven hundred tokens. If the body of `SKILL.md` included these conventions inline, every activation of the skill would pay that cost. With the conventions in a separate reference, the cost is paid only when the body explicitly asks for it, which is once per scaffolding operation rather than on every prompt where the skill is even considered. The savings compound across the entire skill library.

这份 conventions reference 自身约五十行——加载进上下文,要花大约七百 token。若把这些规约直接内联进 `SKILL.md` 的正文,每触发一次 skill 都要付这笔开销;而把它们拆到独立的 reference 里,代价就只在正文显式索取时才付——也就是"每一次脚手架操作"才付一次,而不是"每一个 prompt 一旦考虑这条 skill"都付。这笔节省会在整个 skill 库里复利。

> The templates in the `assets` folder are similarly self-contained. The component template might be twenty lines of TypeScript with the component name marked by a placeholder string. The test template might be fifteen lines with the same placeholder. Each template file is small in isolation, but together they add up to a substantial amount of generated code per scaffolding operation, and keeping them as separate files makes them easy to read, easy to edit, and easy to version-control.

装在 `assets` 里的模板,同样各自自给自足。组件模板可能是二十行 TypeScript,组件名用一个占位符串标出;测试模板可能是十五行,也用同一个占位符。每份模板单看都很小,但合起来,每次脚手架操作都会产出一份体量可观的代码;把它们各自拆成独立文件,既好读、也好改、也好做版本管理。

Skill as Small Program · 把 Skill 视为一份小程序

> A property of this design that deserves attention: the skill is now a small program. The `SKILL.md` body is the controller. The conventions reference is the configuration. The asset templates are the data. This is the same architectural shape that has organized successful software systems for decades. The configuration is separate from the code. The data is separate from the logic. The components are independently meaningful and independently editable. A developer who needs to update the conventions edits one file. A developer who needs to add a fifth template adds one file. A developer who needs to change the procedure edits the body. None of these changes ripples into the others, because the boundaries between the components are clean.

这种设计有一条值得注意的性质:这条 skill 现在已经是一份*小程序*。`SKILL.md` 的正文是"控制器";conventions reference 是"配置"; `assets/` 里的模板是"数据"。这正是过去几十年来组织成功软件系统的同一种架构形态:配置与代码分离、数据与逻辑分离;各组件各自独立成立,各自独立可改。一位开发者要更新规约,改一个文件;要加第五份模板,加一个文件;要修改流程,改正文。这些改动,没有任何一份会波及到其他部分——因为各组件之间的边界清清楚楚。

```
- The component is the default export of its main file.
- The folder's index.ts re-exports the component as a named export so
  that imports can use either form.

## Tests

- Use React Testing Library, not Enzyme.
- Test rendering, user interactions, and accessibility.
- Mock external dependencies with `vi.mock` (or `jest.mock` if the
  project uses Jest).
```

[注:代码块原文保留,以下为本页中译补充]

这是 `references/component-conventions.md` 的剩余部分。"## Exports"小节规定:组件是主文件的 default export,文件夹的 `index.ts` 把它再以 named export 重导——这样上层既可写 `import Button from '.../Button'` 也可写 `import { Button } from '.../Button'`。"## Tests"小节规定:用 React Testing Library(不要用 Enzyme)、要覆盖渲染、用户交互与可访问性、外部依赖用 `vi.mock`(用 Jest 的项目改 `jest.mock`)——这一行同时容纳 Vitest 与 Jest 两种生态,也是把"项目差异"显式编码进 reference 的好例子。

← Chapter 7 上一页

后一页 →

This architectural cleanliness is not an accident, and it is not unique to this skill. The three-folder convention, scripts and references and assets, exists precisely because it produces this kind of clean separation. A skill that follows the convention is a skill that has been organized for editability, for review, and for evolution over time. Skills that ignore the convention can still work, but they tend to be harder to maintain, harder to share, and harder to extend, because the responsibilities are tangled together in ways that make any single change risk affecting things that should be independent of it.

这种架构上的整洁,既不是巧合,也不是这条 skill 所独有。`scripts`、`references`、`assets` 这三目录的惯例之所以是现在这个样子,正是因为它能产生这种"干净分离"。遵守这套惯例的 skill,是为"可改、可 review、可演化"而组织起来的;不遵守的 skill 也能跑,但通常更难维护、难分享、难扩展——因为职责纠缠在一起,任何一次单点改动,都有可能误伤到本应独立的部分。

A final note for this chapter concerns the path forward for any individual skill. Most skills do not start as multi-file skills. They start as single-file skills, often less than fifty lines long, written in fifteen minutes to address a specific friction. Over time, as the developer uses the skill and notices its rough edges, the skill grows. Some of the growth fits naturally inside `SKILL.md`. Some of the growth pushes outward, into reference files when the developer wants to add long-form documentation, into asset files when the developer wants templates, into scripts when the developer wants to delegate deterministic work to real code. The multi-file structure is the destination, not the starting point. Skills that grow toward it gradually, in response to actual need, end up with structures that fit their actual use. Skills that begin life as elaborate multi-file constructions before any real use has been made of them often turn out to have been over-engineered, with bundled resources that nobody ever loads and templates that never get applied to real work. The discipline is to start simple, use the skill, and add complexity only when the skill itself is asking for it.

本章的最后一段,留给"一条 skill 应当走一条什么样的成长路径"这件事。大多数 skill *不是*从多文件起步的;它们从单文件起步——通常不到五十行,用十五分钟写出来,只为治一处具体的摩擦。一段时间过去,开发者不断用、不断注意到它的"毛边",skill 才开始长。有一部分的"长"自然装得下 `SKILL.md`;另一部分"长"会往外挤——加 reference 文件,放长篇文档;加 asset 文件,放模板;加 script 文件,把确定性的活交给真代码。多文件结构是*目的地*,不是出发点。沿着"对真实需要的回应"逐渐走到那里的 skill,最终的结构与它真实的用法严丝合缝;反过来,还没真正被用过就先把多文件结构搭得花里胡哨的 skill,最后几乎都被证明是过度工程——打包的资源无人加载、模板从未被套到真正的工作上。纪律是:从简起步,先把它用起来,等它*自己*开口要更复杂时,再加复杂度。

A useful test for whether a skill is ready to expand into multiple files is the printout test. Print the `SKILL.md` body on paper. Read it through. If, while reading, the eye keeps wanting to skip past large chunks of detail because those chunks are not relevant to the immediate task, those chunks are candidates for relocation into reference files. If, while reading, the body begins to feel less like a procedure and more like a manual, the body has grown too large and needs to be split. Conversely, if the body fits comfortably on a single printed page and reads as a clean

> procedure from beginning to end, the skill does not yet need to expand. The printout test is informal but reliable, and it spares the developer from the trap of expanding skills before they have earned the right to expand.

判断一条 skill "是否已经准备好要扩成多文件"时,有一个好用的测试——"打印测试"。把 `SKILL.md` 的正文打印到纸上,从头读到尾。读的时候,如果眼睛老想跳过"大块的、与当下任务无关的细节"——那些大块,就是"可以搬到 reference 文件里去"的候选;如果读着读着,这份正文已经不再像"流程",而越来越像"手册"——那就说明它已经长得太大,需要被拆。反过来,如果整份正文舒舒服服地印在一张 A4 上,从头到尾读起来都像一份干净的流程,那这条 skill*还不到*需要扩的时候。"打印测试"看似不正式,却很可靠;它能让开发者躲过"在 skill 还没挣得那份复杂度之前就先把它复杂化"的陷阱。

← Chapter 7 上一页

CHAPTER EIGHT →

> The chapter has covered the structural facts. The next chapter takes the natural next step. With the ability to build skills that span multiple files, the developer has the raw material for a library. The question becomes how skills behave when more than one of them might apply to a given request, how they compose with each other when their work overlaps, and how to design libraries in which skills cooperate rather than collide. Composition is what turns a collection of skills into a system, and it is the subject of the chapter ahead.

本章,讲的是"结构事实"。下一章走下一步自然而然的台阶——既然已经能搭出跨多文件的 skill,开发者手里就有了建造"skill 库"的原材料。接下来的问题变成:当一条请求可能同时适用多条 skill 时,它们会怎么表现?当它们的活彼此重叠时,它们之间会怎么组合?怎样设计,才能让一个 skill 库里的各条 skill *彼此协作*而不是*彼此撞车*?组合,是把"一堆 skill"变成"一套 skill 系统"的那一步——而这,正是下一章的主题。

← Chapter 7 结尾

CHAPTER EIGHT →

CHAPTER EIGHT · 第八章
Skill Composition and Collision: When Skills Work Together
Skill 的组合与撞车:让 Skills 协作起来

> *"The library is more than the sum of the books on its shelves only when the shelves themselves have been arranged with thought."*
>
> — Jorge Luis Borges, paraphrased

"图书馆大于'它架上各书之和'这件事,只在'架子本身也是经人琢磨摆出来的'时候才成立。"

—— 豪尔赫·路易斯·博尔赫斯(转述)

A single skill, well written, improves a developer's daily life. A library of skills, well composed, transforms it. The transformation does not happen automatically. Skills do not naturally cooperate. Left to their own devices, skills can also collide: two skills that both think they apply to the same request, with overlapping descriptions, produce inconsistent firing and unpredictable behavior. The discipline of skill composition is the discipline of designing libraries in which the skills clearly belong to their own territories, hand off cleanly when work crosses a boundary, and amplify each other when more than one is needed for a single task. This chapter is about that discipline.

一条写得到位的 skill,能让开发者的日常稍好一点;而一套"组合得道"的 skill 库,会把这种"好一点"升格为"焕然一新"。而这种升格,并不是自动发生的。Skill 之间并不天然协作;若放任自流,它们甚至会撞车——两条 description 有重叠的 skill,都可能认为自己适用于同一条请求,于是触发变得不一致、行为变得不可预测。"Skill 组合"这门手艺,正是"设计一个库:让每条 skill 各自清楚属于自己那片领土,在工作跨界时干干净净地交接,且当一条任务需要多条协作时彼此放大"的纪律。这一章,就讲这套纪律。

Begin with how Claude actually decides which skill to fire. When a user submits a prompt, Claude evaluates the prompt against the descriptions of every available skill. Each description is a piece of evidence about whether the corresponding skill is relevant. Claude integrates the evidence and makes a judgment. In the simplest case, only one skill's description matches the prompt, and that skill fires. In more interesting cases, two or more skills' descriptions match, and Claude must choose. The choice is influenced by how confidently each description seems to match, by how specific each description is, and by any boundary language the descriptions contain. When two descriptions are equally plausible, the firing becomes a judgment call that may go either way on different sessions, which is the practical face of skill collision.

先讲 Claude 实际是怎么决定"哪条 skill 该触发"。当用户提交一条 prompt,Claude 会拿这条 prompt 去与"每条可用 skill 的 description"比对。每条 description 是一份"该 skill 是否相关"的证据;Claude 把这些证据合到一起,做出一份判断。最简单的情况下,只有一条 skill 的 description 与 prompt 匹配,那条就触发;更有趣的情形是:有两条或更多 description 都匹配,Claude 必须从中选一个。选哪条,取决于每条 description "看起来有多像那么回事"、"有多具体"、以及里面是否带边界语言。当两条 description 看起来都差不多合理时,"哪条该触发"就变成了一份"判官题"——可能在不同 session 里结果不同,这正是 skill 撞车在实践中的面相。

Collision is bad not because the wrong skill fires, although that is annoying. Collision is bad because the developer cannot rely on which skill will fire, and reliability is the entire point of installing skills in the first place. A skill library in which firing is reliable is a library that the developer

撞车之所以糟糕,不是因为"错的 skill 被触发"——虽然那确实让人烦。撞车之所以糟糕,是因为开发者无法信任"究竟哪条会触发"——而*可靠性*,正是当初装这套 skill 的全部意义。一个触发可靠的 skill 库,是那位开发者——

← Chapter 7 结尾

后一页 →

can trust. A library in which firing is uncertain is a library the developer must verify on every use, which negates the value of having installed the skills at all.

能放心去信的库;而一个触发不可预期的库,逼着开发者每次用都得亲自去核验——那也就把"当初装 skill 想要的那份价值"全数抹掉了。

Three patterns produce most collisions in real skill libraries. The first pattern is overlapping topic language. Two skills mention the same topic in their descriptions, and Claude cannot tell which one a given request belongs to. A `pr-description` skill and a `pr-review` skill both mention pull requests, and a vague request such as `help me with this PR` could plausibly fire either one. The fix is boundary language: each description should explicitly state what its skill is and is not for. The `pr-description` skill says, `this skill is for writing the`

> `description of a PR you are creating, not for reviewing one`. The `pr-review` skill says, `this skill is for reviewing PRs that someone else wrote, not for drafting your own`. Each description now contains both an inclusion criterion and an exclusion criterion, and the collision evaporates.

真实 skill 库中的大多数撞车,来自三种模式。第一种——"主题措辞重叠"。两条 skill 的 description 都提了同一个主题,Claude 分不清一条给定请求该归哪边。比如 `pr-description` 和 `pr-review` 都会提到 pull request;而一条含混的请求——比如 `help me with this PR`——按理两边都可能被触发。修法是"边界语言":每条 description 都应显式讲清"自己干什么、不干什么"。`pr-description` 这样写: `this skill is for writing the description of a PR you are creating, not for reviewing one`; `pr-review` 这样写: `this skill is for reviewing PRs that someone else wrote, not for drafting your own`。每条 description 此刻都同时携带一条"包含准则"和一条"排除准则",撞车就此散去。

> The second pattern is mismatched specificity. One skill has a very specific description; another has a very general description that happens to overlap with it. The general one tends to win in cases where the specific one should have. The fix is to either make the general skill more specific by adding boundary language, or to recognize that the general skill is poorly designed and should be split into several more focused skills. A general skill called `code-quality` that fires on anything related to code quality is almost always a mistake. It would compete with a `refactor-helper` skill, a `code-review` skill, a `static-analysis` skill, and several others that overlap with parts of its territory. The right response is usually to dissolve the general skill into the specific ones, each of which has a clear description.

第二种——"具体度不匹配"。一条 skill 的 description 写得很具体;另一条的 description 写得很泛,偏偏又与前者有重叠。结果往往是:那条"泛的"在很多本来该"具体的"触发的场景下抢了先。修法有两条:一是用边界语言把那条"泛的"再收紧一些;二是承认那条"泛的"在设计上本就是错的,应当被拆成若干条更聚焦的 skill。一条叫 `code-quality`、凡提到"代码质量"就触发的"通用" skill,几乎永远是个错误——它会与 `refactor-helper`、`code-review`、`static-analysis` 等多条 skill 抢地盘,且抢的范围会与它们各自领域的不同片段都重叠。正确的处理,通常是把它"消解"回那些更具体的 skill,让每一条都各持一份清晰的 description。

Naming Drift · 命名漂移

> The third pattern is naming drift. The skill author writes the description in one vocabulary and the body in another. The user, naturally, employs vocabulary that matches the body more than the description, because the body is closer to the actual work. The skill under-fires because the description does not contain the words the user actually uses. The fix is to bring the description into alignment with the working vocabulary of the body. A useful exercise is to read the body of the skill, list the ten words that appear most frequently and seem most central to what the skill does,

第三种——"命名漂移"。Skill 作者写 description 时用的是一套词,写正文时用的是另一套;用户自然偏向使用"更贴正文"的那套词——因为正文更靠近实际工作。结果就是 skill 欠触发:description 里没装用户真正在用的那些词。修法是把 description 拉回到与正文的"工作词表"对齐。一条好用的练习:把 skill 正文读一遍,列出"出现频率最高、对描述 skill 实质而言最关键"的十个词,

← Chapter 8 上一页

后一页 →

> and check that those words appear in the description. If they do not, the description should be rewritten to include them.

再核对一遍,看这十个词有没有出现在 description 里。如果没有,就该把 description 改写,把这些词补上。

With the failure modes named, turn to composition itself. Composition is the positive case: two or more skills cooperating on a single request to produce something better than either could produce alone. Several patterns of composition are worth knowing.

失败模式已经命名完,接下来看组合本身。所谓"组合",是正面情形——两条或更多条 skill 协作完成同一条请求,产出的东西比任何一条单干都要好。值得了解的有几种组合模式。

The first composition pattern is sequential composition. Two skills fire in sequence on the same request, with the output of the first becoming the context for the second. A common case: the user asks for a refactoring plus a commit. The `refactor-helper` skill fires first, executes the refactor, and produces a description of what changed. The `commit-message` skill fires next, reads the changes, and produces a commit message that accurately reflects them. Sequential composition often happens implicitly: the developer asks Claude to do two things in one prompt, and Claude activates both relevant skills as it works through the prompt. The skill author can encourage this by writing skill bodies that conclude with a clean handoff. The `refactor-helper` skill, after completing its work, might end its body with a line such as `if the user wants a commit message for these changes, the commit-message skill is appropriate`. Claude reads that hint and is more likely to fire the second skill at the right moment.

第一种组合模式——"顺序组合"。两条 skill 在同一条请求上按序触发,第一条的输出成为第二条的上下文。常见情形:用户要求"重构 + commit"—— `refactor-helper` 先触发,把重构做完,产出一段"改了什么"的说明; `commit-message` 紧接着触发,读入那些改动,产出一条准确反映它们的 commit message。顺序组合常常是隐式发生的:用户在一个 prompt 里让 Claude 干两件事,Claude 在推进过程中把两条相关的 skill 都激活了。Skill 作者可以"鼓励"这种发生——办法是在正文末尾留一句干净的"交接语"。 `refactor-helper` 干完活之后,可以在正文结尾留一句 `if the user wants a commit message for these changes, the commit-message skill is appropriate` 。Claude 读到这一句,就更可能在合适的时刻触发第二条 skill。

The second composition pattern is parallel composition. Two or more skills fire on the same request and contribute different perspectives to the same output. A code-review request might activate a security-focused skill, a performance-focused skill, and a style-focused skill simultaneously, with the final review weaving together all three perspectives. Parallel composition is more common when the request is broad and the skills are narrow, because each narrow skill contributes its own piece of the broad answer. The skill author encourages parallel composition by writing skills that produce well-bounded, additive output rather than total reports. A security-focused review skill that produces a self-contained section labeled `Security` and stops there composes well with other focused skills. A security-focused review skill that produces an entire end-to-end review report does not, because its output overlaps with what the other skills want to produce.

第二种组合模式——"并行组合"。两条或更多条 skill 在同一条请求上同时触发,为同一份输出贡献各自不同的视角。一条 code-review 请求,可能同时激活一条安全聚焦的 skill、一条性能聚焦的 skill、一条风格聚焦的 skill;最终的那份 review,把三者的视角编织在一起。并行组合更常出现在"请求很宽、skill 都很窄"的情形下——因为每条窄 skill 只贡献宽答案里的自己那一块。Skill 作者要鼓励并行组合,办法是把 skill 写成"产出边界清晰、彼此可叠加"的输出,而不是"一份大全的报告"。一条产出"名为 `Security` 的自成一节、到那里就停"的安全 review skill,很能与其它聚焦的 skill 组合;而一条产出"端到端完整 review 报告"的安全 review skill,就组合不动——因为它的输出会与其它 skill 想要产出的东西撞上。

← Chapter 8 上一页

后一页 →

The third composition pattern is the meta-skill. A meta-skill is a skill whose job is to orchestrate other skills. It has a description that fires on broad requests, and its body contains instructions that direct Claude to invoke specific other skills in a specific order. Meta-skills are useful for workflows that recur often enough to deserve a single trigger but that involve several distinct steps. A `release-prep` meta-skill might fire on a

> request to prepare a release and direct Claude to invoke a `changelog-generator` skill, a `version-bump` skill, a `release-notes` skill, and a `tag-and-push` skill, in sequence. The meta-skill is a small orchestrator, and the actual work is done by the focused skills it directs.

第三种组合模式——"元 skill"。所谓元 skill,是一条专门用来"指挥其它 skill"的 skill:它的 description 在较宽的请求上触发;它的正文里装着若干指令,告诉 Claude 按某种顺序去调用哪些具体 skill。元 skill 适用于"频繁到值得为它设一个单一触发点、但内部又包含几个互不相同步骤"的 workflow。比如一条叫 `release-prep` 的元 skill,会在"准备发版"的请求上触发,指引 Claude 依次调用 `changelog-generator` skill、`version-bump` skill、`release-notes` skill 和 `tag-and-push` skill。元 skill 自己是个小调度者,真正干活的,是它所指引的每一条聚焦 skill。

A Practical Guideline · 一条实操准则

> A practical guideline ties the patterns together. Build narrow skills first. A library of twenty narrow, focused skills is more useful than a library of five general ones. Narrow skills fire reliably because their descriptions can be precise. Narrow skills compose because they have well-bounded outputs. Narrow skills are easier to debug because their failure modes are concentrated. The temptation to build broad skills usually comes from a desire to reduce the number of skills in the library, but this is the wrong objective. The right objective is to maximize reliability and composability, both of which point toward narrow skills.

一条实操准则,把这几种模式串到了一起——先搭窄的 skill。一个装二十条"窄而聚焦" skill 的库,比一个装五条"宽而泛" skill 的库要有用得多。窄 skill 触发得稳,因为它们的 description 能写得很精确;窄 skill 组合得动,因为它们的输出边界清楚;窄 skill 容易 debug,因为失败模式集中在一处。"想搭宽 skill"的诱惑,通常来自"想让库里少几条"的愿望——但那是个错误的目标。正确的目标是"最大化可靠性 + 可组合性",这两条都指向窄 skill。

> A second guideline concerns the boundary between skills and the body of `CLAUDE.md` . Some knowledge belongs in `CLAUDE.md` , where it is always loaded. Some knowledge belongs in a skill, where it is loaded on demand. The dividing line is roughly: knowledge that is true at all times in this project belongs in `CLAUDE.md` ; knowledge that is true only when a specific kind of work is being done belongs in a skill. The rule sounds simple, and in practice it requires some judgment, but applying it consistently produces a Claude Code environment where ambient knowledge is always present, targeted knowledge is loaded only when relevant, and the boundary between the two is principled rather than arbitrary.

第二条准则,关于"skill 与 `CLAUDE.md` 正文之间的分界"。一些知识该放在 `CLAUDE.md` ——它会一直被加载;另一些知识该放进 skill——按需加载。粗略的划分线是:在本项目里*任何时候都成立*的知识,放 `CLAUDE.md` ;只在*某类具体工作进行时才成立*的知识,放 skill。这条规则听上去简单;实操中确实需要一些判断;但只要能一以贯之地用,就能造就一个这样的 Claude Code 环境——"环境性"的知识永远在场,"针对性"的知识只在相关时入场,两者之间的边界,是有原则的、不是任意的。

> There is one more compositional possibility worth naming, although its full treatment belongs in Part Three. Skills can be invoked by subagents as well as by the main session. A subagent dispatched for a specific kind of work inherits the skill library available in the project and can activate any

还有一种组合上的可能性值得点出名字——虽然完整的展开属于第三部分。Skill 也能被 subagent 调用,而不只是被主会话调用。被派去处理某类具体工作的 subagent,会继承项目里那份可用的 skill 库,并能激活其中的任一

← Chapter 8 上一页

后一页 →

skill whose description matches the work the subagent is doing. This is a powerful pattern, because it lets the skill library and the subagent library reinforce each other. A `test-writer` subagent that has access to a project's test-conventions skill is more reliable than either the subagent or the skill alone. The full implications of this composition will be developed in Chapter Fourteen, after subagents have been introduced. For now, it is enough to know that the skills built in this part of the book will continue to do work even after the boundaries of skills as a topic are crossed.

条 description 与该 subagent 正在做的工作匹配的 skill。这是一股很强的力量——它让 skill 库与 subagent 库彼此强化。一个持有"本项目测试规约" skill 访问权的 `test-writer` subagent,比 subagent 或 skill 任何一方单独存在时都更可靠。这种组合的全部内涵,会在第十四章——也就是介绍完 subagent 之后——再展开。此刻只需要知道一件事:这一部分搭出的 skill,在我们跨过"skill 作为一个话题"的边界之后,仍会继续工作。

This chapter has dealt with the politics of a skill library: how skills relate to each other, how they collide and how they cooperate, and what the author can do to encourage the second over the first.

本章讲的是"一个 skill 库内部的'政治学'"——skill 之间如何相互关联、如何撞车、如何协作,以及作者可以做些什么,来鼓励后一种情况、多于前一种。

A practical exercise will reinforce the principles. Imagine a developer who, over six months of casual skill building, has accumulated a library of fifteen skills. She has noticed that some of them seem to fire reliably and others fire inconsistently. She decides to audit the library against the principles in this chapter.

一份实操练习,用来强化这些原则。设想一位开发者,经过六个月的随手积累,手上已经攒起了十五条 skill。她注意到其中一些触发得很稳,另一些却忽好忽坏;于是决定按本章的原则,给这个库做一次"审计"。

The Audit Procedure · 审计流程

The audit procedure is straightforward. She lists the fifteen descriptions side by side. She reads each one and scores it against the four-part framework from Chapter Six: canonical statement, trigger language, boundary language, input expectation. She also reads each pair of descriptions and asks whether either pair could plausibly fire on the same request. Of the fifteen skills, three pairs jump out as potential collisions. A `pr-description` skill and a `pr-review` skill, both mentioning pull requests. A `code-explain` skill and an `explain-codebase` skill, both mentioning code explanation. A `bug-report` skill and a `debug-helper` skill, both mentioning bugs.

审计流程本身很直白。她把十五条 description 一字排开,逐条对照第六章的"四段式框架"来打分——典型陈述、触发语言、边界语言、输入预期。同时,她把每两条 description 配对读,问自己:"这一对,有没有可能在同一条请求上被同时触发?"十五条之中,有三对明显撞上:`pr-description` 与 `pr-review` ——都提到 pull request;`code-explain` 与 `explain-codebase` ——都提到代码解释;`bug-report` 与 `debug-helper` ——都提到 bug。

Each pair gets resolved differently. The first pair is the cleanest case. Each skill clearly has a different job, but neither description names the difference explicitly. She adds a single sentence of boundary language to each. The `pr-description` description gains the line, `this skill is for writing the description of a PR you are creating, not for reviewing one`. The `pr-review` description gains the symmetrical line, `this skill is for reviewing PRs that someone else wrote, not for drafting your own`. The two skills now have non-overlapping territories. Subsequent firing becomes reliable.

每对的修法各不相同。第一对是最干净的情况:两条 skill 的活明显不同,但没有一条 description 把这种不同说清。她在两边各加一句边界语言。`pr-description` 的 description 加上这么一句:`this skill is for writing the description of a PR you are`

`creating, not for reviewing one`;`pr-review` 的 description 加上对称的那一句:`this skill is for reviewing PRs that someone else wrote, not for drafting your own`。两条 skill 此刻的领域不再重叠;此后的触发就稳了。

← Chapter 8 上一页

后一页 →

> The second pair is more interesting. Reading them carefully, she realizes that `code-explain` and `explain-codebase` are not really separate skills. `code-explain` operates on a piece of code that has been pasted or pointed to. `explain-codebase` operates on a whole repository. The original intent was that they would handle different scales of explanation. But the descriptions have not made the scale distinction clear, and in practice users have been triggering whichever description Claude happened to match first, regardless of which scale the user actually wanted. She rewrites both descriptions to make the scale explicit. `code-explain` says, `explains a single function, file, or pasted snippet`. `explain-codebase` says, `produces a high-level architectural summary of an entire codebase`. Now the descriptions are not just non-overlapping; they are explicitly complementary. A user who pastes a function gets `code-explain`. A user who asks for a tour of the repo gets `explain-codebase`. The collision evaporates.

第二对,更有意思一些。仔细一读,她意识到 `code-explain` 与 `explain-codebase` 实际上*不是*两条彼此独立的 skill:`code-explain` 处理的是"被粘贴或被点出"的一段代码;`explain-codebase` 处理的是整个仓库。原本的设想,是让它们分别应付不同尺度的"解释";但 description 里没把这种尺度差讲清,实际跑下来,用户触发的常常是"Claude 先碰上的那一条"——无论用户真正想要的是哪种尺度。她把两边的 description 都改写,把"尺度"讲得明确。`code-explain` 写:`explains a single function, file, or pasted snippet`;`explain-codebase` 写:`produces a high-level architectural summary of an entire codebase`。此刻的 description 不再只是"不重叠",而成了"显式互补"——粘一段函数的用户拿到 `code-explain`;想要"逛一下这个 repo"的用户拿到 `explain-codebase`。撞车散去。

> The third pair turns out to be a deeper problem. The `bug-report` skill formats a structured bug report. The `debug-helper` skill walks the user through diagnosing a bug. Both fire on requests that mention bugs. After looking at how she actually used the two skills over the past month, she realizes that `debug-helper` has fired roughly four times, all of them in cases where `bug-report` would also have been a reasonable choice, and none of those four sessions produced output she actually used. `debug-helper`, in retrospect, was an attempt to build a skill that did too much: a generic debugging companion. Its description was broad because its job was broad, and broad jobs do not fit the skill model well. She decides to retire `debug-helper` and replace it, eventually, with two or three narrower skills that cover specific debugging scenarios. For now, removing it cleans up the library and removes the collision with `bug-report`. The library shrinks from fifteen skills to fourteen, and quality goes up rather than down.

第三对,挖出来的是一个更深的问题。`bug-report` skill 把一条 bug 报告整理成结构化形式;`debug-helper` skill 引导用户走完"诊断 bug"的流程。两条都会在"提到 bug"的请求上触发。回看过去一个月里她对这两条 skill 的实际使用,她发现 `debug-helper` 大约触发了四次——四次之中,每次其实由 `bug-report` 来跑也合理;且这四次产出的东西,她事后都没真的用过。回头看,`debug-helper` 是一次"想搭一条能干太多事的 skill"的尝试——做一个"通用的 debug 伙伴"。它的 description 之所以宽,是因为它的活本来就很宽;而那种"宽活"和 skill 这个模型本身就不合身。她决定把 `debug-helper` 退下来,日后用两三条更窄的 skill 去覆盖具体的 debug 场景;眼下这一撤,既清理了库,也消除了与 `bug-report` 的撞车。库从十五条缩到十四条,质量却反过来上升了。

> This kind of audit is worth performing once a quarter on any skill library that has grown past ten skills. The principles are simple: list the descriptions, score them against the four-part framework, look for collisions, and either resolve the collisions through boundary language, refactor through clearer scope language, or retire skills that have proven to be too broad to do their job well. A library that has been audited recently is a library that the developer can trust, and trust is what makes the library worth using.

这种审计,值得在任何一个超过十条 skill 的库上,每季度跑一次。原则很简单:把 description 摆出来,按"四段式框架"逐条打分,寻找撞车;每一起撞车,要么用边界语言解决,要么用更明确的 scope 措辞重构,要么把那些"宽到干不好活"的 skill 直接退掉。一个最近被审计过的库,是一个开发者能信得过的库;而*信得过*,正是让这个库"值得用"的那份东西。

← Chapter 8 上一页

CHAPTER NINE →

Composition Across Domains · 跨领域的组合

A second concrete scenario, which surfaces a different design pattern, involves a developer building a skill library for a team. The team has a shared `.claude/skills/` directory in their project, and three developers are contributing to it. Each developer has, naturally, drafted a few skills in her own style. The result, after a month, is a library that is technically functional but stylistically inconsistent. Some descriptions follow the four-part framework. Others are one-liners. Some bodies use the step-numbered procedure pattern. Others use prose. Some skills assume the user will provide context. Others have built-in steps to gather it.

第二份具体的场景,展示的是一种不同的设计模式——一位开发者为一个团队搭一个 skill 库。团队在项目里共享一份 `.claude/skills/` 目录,三位开发者各自向里面贡献。自然地,每个人都按自己的风格起草了几条 skill。一个月之后,这个库在技术上能跑,但风格上一团混乱:有些 description 走"四段式",有些只是一句话;有些正文用"步骤编号"的流程,有些用的是散文;有些 skill 假定用户会主动提供上下文,有些则自带"收集上下文"的步骤。

The team decides to standardize. They sit down together for thirty minutes and agree on a small set of conventions. Every description must include canonical, trigger, boundary, and input expectation language. Every body must use a step-numbered procedure for any skill whose work is sequential. Every skill that needs context must include a step-one section that gathers that context, rather than relying on the user to provide it pre-emptively. The conventions are written into a short `README.md` at the root of the `.claude/skills/` directory, so that any future contributor encounters them at the moment she opens the directory for the first time.

团队决定把风格统一起来。他们坐下开了三十分钟的会,就一套小规模的约定达成一致:每条 description 必须含典型陈述、触发语言、边界语言、输入预期四段;每条正文,凡是工作本身是顺序性的,就必须用"步骤编号"流程;每条需要上下文的 skill,都必须自带一个"第一步:收集上下文"小节——而不是指望用户主动预备好。这些约定被写进一份放在 `.claude/skills/` 根目录的简短 `README.md`,于是任何一位未来的贡献者,在她第一次打开这个目录的那一刻,就会与这份约定不期而遇。

The conventions take effect immediately for new skills, and existing skills are gradually retrofitted as they need attention for other reasons. Within a month, the library has a coherent house style. Skills feel like they belong together. New contributors can write a new skill in fifteen minutes because the conventions answer most of the design questions before the contributor even asks them. The library has become, in the genuine sense of the word, a library, with a unifying organizing principle, rather than a heap of files in a folder.

约定对新搭的 skill 立刻生效;已有的 skill,借着"出于别的原因需要被关注"的机会,被逐条回炉。不到一个月,这个库就有了"一种连贯的自家风格"——skill 们看上去"彼此是一伙的"。新来的贡献者能在十五分钟内写出一条新 skill,因为那些"原本要靠问才能答出来"的设计问题,已经被约定提前答好。这个库,在最严格的意义上,成了一座*图书馆*——带着一种统一的组织原则,而不只是"文件夹里的一堆文件"。

A final thought on composition before this chapter ends. The most powerful skill libraries in the wild, the ones that earn the title library rather than mere collection, share a property that is worth naming. They are designed for the user's actual workflow, not for the developer's sense of

> what skills should exist. A skill that addresses a friction the developer has experienced personally, in real work, will almost always fit cleanly into a library. A skill that the developer built because she imagined someone might want it, without having needed it herself, will almost always feel out

本章收尾前,关于"组合"再留一句观察。野外最强大的那些 skill 库——那些真配得上"library"二字、不只是"collection"的——共有一条值得点出的性质:它们是*按用户的实际 workflow*来设计的,而不是按"开发者自己觉得应该有哪些 skill"来设计的。一条 skill,处理的是开发者在真实工作中亲自撞过的那处摩擦,几乎总能干干净净地装进一个库;一条 skill,是开发者*想象*"也许有人想要"、但自己并没用过的那种,几乎总会在库——

← Chapter 8 上一页

CHAPTER NINE →

> of place. The audit principle that follows from this is to be ruthless about skills that have not been used. A skill that has not fired in three months is a skill that does not belong in the library, even if its description is technically excellent. The cost of keeping it is small but real, and the noise it adds to the firing decision marginally degrades every other skill that competes for the same attention.

里显得格格不入。从这条观察里,直接派生出的一条审计原则是:对"长期没被用过"的 skill,要狠得下心。一条三个月都没触发过的 skill,即使 description 在技术上堪称漂亮,也不该再留在库里。留着它的代价,虽小但真实;它给"该触发哪一条"这件事多加的噪声,会让每一条与它争抢注意力的 skill 都被稍微拉低一点。

> A pattern that separates great libraries from merely good ones is the deliberate cultivation of trigger overlap with the developer's natural vocabulary. The developer who pays attention notices, after a few months, that she has habits of phrasing that recur. She tends to say `make me a`, when she wants something generated. She tends to say `walk me through`, when she wants something explained. She tends to say `tighten this up`, when she wants something refined. Each of these recurring phrasings is a hook, and skills whose descriptions explicitly include the developer's recurring phrasings fire more reliably than skills whose descriptions use generic alternatives. Building this kind of personal-vocabulary alignment into descriptions is a small practice, but it converts a library from one that fires correctly to one that fires effortlessly. The library starts to feel less like a tool the developer is using and more like an extension of her own way of speaking. This is the felt sense of a mature library, and it cannot be achieved on the first day. It is the gradual product of attention paid to the developer's own habits, applied iteratively across descriptions over time.

"伟大的库"与"仅仅好的库"之间,有这样一种"会区分"的模式:主动去让 description 与开发者的"自然词表"产生触发重叠。一位留心的开发者,几个月后就会注意到自己反复出现的措辞习惯——她想"生成"什么时,习惯说 `make me a`;她想"讲解"什么时,习惯说 `walk me through`;她想"打磨"什么时,习惯说 `tighten this up`。每一种这样的"反复措辞",都是一条钩子;哪些 skill 的 description 显式包含这些钩子,触发起来就比"用通用替代措辞"的描述更稳。把"个人词表对齐"这层动作嵌进 description,是一件小习惯;但它能把一个库从"触发得对"升级到"触发得毫不费力"。这个库会开始"不太像"开发者正在使用的一件工具,而"越来越像"她说话方式的一种延伸。这就是"一座成熟库"的那种体感——它不是第一天就能达到的;它是开发者把注意力持续放在自己的措辞习惯上、一次又一次把"对齐"应用到 description 里,日积月累的产物。

> The next chapter shifts from politics to inventory. With the principles of composition in hand, the reader will build a personal library of ten skills that are immediately useful in daily work, with each skill written, debugged, and ready to install. The chapter ahead is the most practical in the book. By its end, the reader's Claude Code environment will be measurably more capable than it was at the start of Part Two, and the value of the principles taught in the chapters before it will be visible in every session that follows.

下一章,从"政治学"切换到"清单"——手上握过组合原则之后,读者要亲手搭起一份"十条 skill 的个人库",每一条都"立刻就能在日常工作里用得上",且每一条都写好、调好、装好就位。下一章是全书最"实操"的一章。读完之后,读者的 Claude Code 环境,在能力上会有一种"可被量化"地提升——而前面几章所教的那些原则,会在此后每一个 session 中以可见的方式兑现。

← Chapter 8 上一页

CHAPTER NINE →

CHAPTER NINE · 第九章

The Personal Skill Library: Your First Ten Skills

个人技能库:你的前十个技能

> *"A library is not a collection of books. A library is a set of decisions about which books to keep at hand."*
>
> — Umberto Eco, paraphrased

"图书馆不是藏书的集合,图书馆是一组关于'哪些书该放在手边'的决定。"

—— 安伯托·艾柯(转述)

This chapter is the most practical in Part Two. By the time the reader finishes it, ten new skills will be installed in her Claude Code environment, each of them ready for immediate use, each of them solving a specific recurring friction in daily development work. The skills are presented in a uniform format. Each one begins with the problem it solves, then shows the complete `SKILL.md` file ready to copy, then explains the design choices that shaped it, then notes the most common pitfall and the recommended way to extend the skill once the developer has lived with it for a while.

本章是第二部分里最"动手"的一章。等读者读完时,她的 Claude Code 环境里会多装十个新 skill——每一个都马上可用,每一个都解决日常工作里某一种反复出现的摩擦。展示这些 skill 时用的是统一格式:每个 skill 都先讲它解决的问题,然后贴出可以直接拷走的完整 `SKILL.md` 文件,再解释塑造它的设计选择,最后点出最常见的坑,以及开发者与这个 skill 共处一段时间后该如何扩展它。

The selection of ten skills is deliberate. Each skill represents a category of friction common to almost every developer using Claude Code. The categories are: composing structured documents from raw material, generating standardized scaffolding, performing focused review, capturing knowledge from work that just happened, formatting and conforming to project conventions, and coordinating between code and surrounding artifacts such as commits and pull requests. The reader who installs all ten will have her core daily workflow substantially automated. The reader who installs even three or four will feel the difference within a working week.

选这十个 skill 是有讲究的。每个 skill 都对应一类"几乎所有用 Claude Code 的开发者都会遇到"的摩擦。这些类别包括:把原始素材组织成结构化文档、生成标准化脚手架、做有针对性的评审、从刚刚发生的工作中捕捉知识、按项目规约格式化与对齐、以及在代码和周边产物(如 commit、PR)之间做协同。把十个全装上的读者,核心日常工作流会大幅自动化;哪怕只装三四个,也能在一个工作周内明显感到差别。

> Before beginning the build, a small piece of housekeeping. All ten skills in this chapter are intended to be personal skills, installed at the user level, and therefore live in `~/.claude/skills/`. Each one needs its own folder, named to match the skill. The folder structure for the full library will look like this when complete.

开搭之前,先做点内务。本章的十个 skill 都是"个人 skill",按用户级别安装,因此都放在 `~/.claude/skills/` 下。每个 skill 一个目录,目录名跟 skill 名一致。整套库装好之后,目录结构会是这样:

```
~/.claude/skills/
├── commit-message/
├── pr-description/
├── humanize-text/
├── refactor-readability/
├── jest-test/
├── pr-review/
├── code-explain/
├── bug-report/
├── decision-log/
└── changelog-entry/
```

← 前一页 后一页 →

Setting Up Each Skill · 每个 skill 的安装方式

> Each of the ten skills below requires the same setup: create the named folder, create a `SKILL.md` file inside it, paste the contents shown. After installing all ten, restart Claude Code so that the new skills are discovered. From that point forward, the skills will be available in every session.

下面这十个 skill,装法都一样:建一个对应名字的目录,在里面建一个 `SKILL.md` 文件,把展示的内容贴进去。十个都装好后,重启 Claude Code,让新 skill 被发现。从那之后,这些 skill 就会在每个 session 里都可用。

> The first skill is `commit-message`. It was built in detail in Chapter Five and serves as the canonical reference for what a small skill looks like. Its job is to produce conventional commit messages from staged changes. If the reader has not already installed it, the `SKILL.md` file from Chapter Five can be used directly.

第一个 skill 是 `commit-message`。它在第五章里已经详细搭过,这里作为"小 skill 长什么样"的范本。它的职责是:基于已暂存(staged)的改动,产出符合 conventional 规范的 commit message。如果读者还没装,直接把第五章里的那个 `SKILL.md` 拿来用就行。

> The second skill is `pr-description`, used as an example in earlier chapters. It produces pull request descriptions in a consistent format.

第二个 skill 是 `pr-description`,前面的章节里已经作为示例出现过。它的职责是:用统一格式产出 pull request 描述。

```
---
name: pr-description
```

```
description: Drafts a pull request description in the team's standard format. Use whenever the user asks for a PR de
allowed-tools: Bash, Read, Grep
---
# PR Description
When asked for a PR description, follow this procedure.
1. Identify the diff. If the user mentions a branch, run `git log` and `git diff` against the merge base. If no bran
2. Read the diff in full.
3. Produce the description in this exact format:
## Summary
A single sentence in the imperative mood describing the change.
```

> The third skill is `humanize-text`. Its job is to take written content that sounds machine-generated, formal, or stilted, and to rewrite it in a more natural human voice. This skill is especially valuable for developers who write documentation, blog posts, or release notes alongside their code, and who have noticed that AI-generated drafts have a recognizable tone that benefits from editing.

第三个 skill 是 `humanize-text`。它的职责是:把听起来像机器写的、过于正式、生硬别扭的文字,改写成更自然的人类口吻。这个 skill 对那些"除了写代码还要写文档、写博客、写 release notes"的开发者尤其有用——他们早已发现,AI 生成的草稿有一种可辨识的味道,过一遍编辑会好得多。

```
## What changed
A bulleted list of concrete changes, one bullet per logical unit.
## Testing
A short paragraph describing how the change was tested.
## Risk
One sentence categorizing risk as low, medium, or high with a one-line reason.
Do not invent changes that do not appear in the diff. If the diff is large, summarize at the file level rather than
```

```
---
name: humanize-text
description: Rewrites text to sound more human, less formal, and less machine-generated. Use whenever the user asks
allowed-tools: Read
---
# Humanize Text
When asked to humanize a piece of text, apply these transformations.
- Remove em-dashes. Replace with commas, periods, or sentence breaks.
- Remove or replace AI-flavored vocabulary: "delve," "leverage," "robust," "navigate," "in conclusion," "it is impor
- Vary sentence length deliberately. Short sentences punctuate. Long sentences explore. Avoid runs of medium-length
- Cut adverbs that do not earn their place: "really," "very," "quite," "extremely."
- Remove rule-of-three constructions. If a sentence lists three things in parallel for rhythm rather than meaning, r
- Cut filler openers: "It is worth noting that," "It should be mentioned that."
- Use contractions where the surrounding tone permits them.
- Prefer concrete nouns over abstract ones.
Return the rewritten text in a fenced code block. After the block, list the most significant changes you made, no mo
```

← 前一页　　后一页 →

The Refactor Skill · 重构技能

> The fourth skill is `refactor-readability`. Its job is to take a function or section of code that works correctly but reads poorly, and propose a refactor focused on clarity rather than functionality.

第四个 skill 是 `refactor-readability`。它的职责是:拿到一段"功能正确但读起来费劲"的函数或代码块,给出一个*以可读性为目标*的重构方案——但不改它的行为。

```
---
name: refactor-readability
description: Proposes a readability-focused refactor of a given function or section of code, without changing its be
allowed-tools: Read, Edit
---
# Refactor for Readability
When asked to refactor code for readability, follow this procedure.
1. Read the code in full.
2. Identify the top three readability problems. Common candidates: deeply nested conditionals, names that do not com
3. Propose a refactor that addresses those three problems while preserving behavior exactly.
4. Show the refactor as a complete replacement of the original, not as a diff.
5. After the code, list the three problems addressed in plain language.
Do not change behavior. If you notice a bug in the original code, mention it separately, do not silently fix it as p
```

> The fifth skill is `jest-test`. Its job is to write a Jest or Vitest test file for a given function or module, following sensible defaults for both frameworks.

第五个 skill 是 `jest-test`。它的职责是:为一个给定的函数或模块写出 Jest 或 Vitest 测试文件,且对两个框架都使用合理的默认设置。

```
---
name: jest-test
description: Writes a test file in Jest or Vitest for a given function or module. Use whenever the user asks to "wri
allowed-tools: Read, Write
---
# Jest or Vitest Test
When asked to write tests, follow this procedure.
1. Read the file under test.
2. Detect the test framework. If the project uses `vitest` (check package.json), use Vitest. Otherwise default to Je
3. Identify the public surface of the file: exported functions and classes.
4. For each exported function, write tests that cover at minimum: the happy path, one boundary condition, and one er
5. Use `describe` blocks per function and `it` blocks per scenario.
6. Mock external dependencies. Do not make real network calls or write to real files in tests.
7. Place the test file alongside the file under test, with the suffix `.test.ts` (or `.test.js` if the project is Ja
Output the complete test file. Do not abbreviate.
```


> The sixth skill is `pr-review`. Its job is to produce a substantive review of someone else's pull request, complete with concrete suggestions and a summary judgment.

第六个 skill 是 `pr-review`。它的职责是:对别人提的 pull request 做一次有实质内容的评审——既要有具体的建议,也要有总结性判断。

```
---
name: pr-review
description: Reviews someone else's pull request and produces a structured review with comments, suggestions, and a
allowed-tools: Bash, Read, Grep
---
# PR Review
When asked to review a PR, follow this procedure.
1. Identify the PR or diff to review. If the user provides a branch name, diff against the merge base.
2. Read the diff in full before forming any opinions.
3. Group your observations into four sections:
## Substantive comments
Issues that affect correctness, security, or maintainability. Each comment should reference a file and line and expl
## Suggestions
Improvements that are not strictly required but would make the change better. Same format as substantive comments.
## Questions
Things that are unclear from the diff and would require clarification from the author.
## Summary
A two- or three-sentence judgment. Approve, request changes, or defer with reasons.
Do not invent issues. If the change is good, say so plainly. A short positive review is better than a long padded on
```

> The seventh skill is `code-explain`. Its job is to explain a piece of code at a level appropriate to a colleague who knows the language but does not know this codebase. This skill is especially useful when joining a new project, when reading unfamiliar parts of a familiar project, or when preparing notes to share with teammates.

第七个 skill 是 `code-explain`。它的职责是:向一位"懂这门语言但不了解这个代码库"的同事解释一段代码。这个 skill 在以下场景尤其有用:刚加入新项目、读熟悉项目的陌生模块、或者要准备笔记分享给团队成员。

```
---
name: code-explain
description: Explains a piece of code at the level of a colleague who knows the language but not this codebase. Use
allowed-tools: Read
---
# Code Explanation
When asked to explain code, follow this procedure.
1. Read the code in full before writing anything.
2. Identify the purpose of the code in one sentence.
3. Identify the inputs, the outputs, and any side effects.
4. Explain the algorithm or pattern at a high level. Use plain language. Avoid restating the code line by line.
5. Note any non-obvious choices the author made and what alternatives they might have considered.
6. End with a short list of questions you would ask the original author if you could.
Do not pad the explanation with general background. Assume the reader knows the language. Focus on what is specific
```

← 前一页

后一页 →

The Bug Report Skill · Bug 报告技能

> The eighth skill is `bug-report`. Its job is to take a half-articulated description of a problem and produce a structured, actionable bug report that a developer or team could file directly into a tracker.

第八个 skill 是 `bug-report`。它的职责是:拿到一段"没完全说清楚"的问题描述,产出一份结构化、可直接录入 issue tracker 的 bug 报告。

```
---
name: bug-report
description: Turns an informal description of a problem into a structured bug report ready to file in an issue track
allowed-tools: Read
---
# Bug Report
When asked to write a bug report, follow this procedure.
1. Identify what the user has told you and what is missing. If essential information is missing (steps to reproduce,
2. Produce the report in this format:
## Summary
A single sentence describing the bug.
## Steps to reproduce
A numbered list, precise enough that another person could follow them.
## Expected behavior
What should happen.
## Actual behavior
What happens instead.
## Environment
Relevant version numbers, browsers, operating systems, or configurations.
## Notes
Anything unusual, intermittent, or worth investigating.
Stay close to what the user actually said. Do not invent symptoms or steps.
```

> The ninth skill is `decision-log`. Its job is to capture a technical decision the developer just made, in the standard format used for architecture decision records, so that future developers (including the future self) can understand both the decision and its context.

第九个 skill 是 `decision-log`。它的职责是:把开发者刚刚做出的技术决策,按架构决策记录(ADR)的标准格式记录下来,以便未来的开发者(包括未来的自己)同时理解"决策本身"和"它所在的上下文"。

```
---
name: decision-log
description: Records a technical decision in the architecture decision record (ADR) format. Use whenever the user sa
allowed-tools: Read, Write
---
# Decision Log Entry
When asked to record a decision, follow this format.
# ADR-NNN: [Short title in the form of a noun phrase]
## Status
Proposed, Accepted, Superseded, or Deprecated.
## Context
What is the situation that requires a decision? Two or three sentences.
## Decision
What was decided? One or two sentences in the active voice.
## Consequences
What changes as a result? Both positive and negative consequences. A short bulleted list is acceptable here.
## Alternatives considered
What else was on the table, and why were those alternatives rejected? One or two sentences per alternative.
If the user has not specified an ADR number, ask once whether to assign the next sequential number based on existing
```

> The tenth and final skill is `changelog-entry`. Its job is to write a single entry suitable for inclusion in a `CHANGELOG.md` file, given a recent change or set of changes.

第十个、也是最后一个 skill 是 `changelog-entry`。它的职责是:给定一组最近的改动,产出一条适合写入 `CHANGELOG.md` 的单条记录。

```
---
name: changelog-entry
description: Writes a single entry for a project's CHANGELOG.md based on a recent change. Use whenever the user asks
allowed-tools: Bash, Read
---
# Changelog Entry
When asked for a changelog entry, follow this procedure.
1. If the change is described in conversation, use that. If a commit or branch is referenced, read the diff to under
2. Categorize the change as one of: Added, Changed, Fixed, Removed, Deprecated, Security.
3. Write a single line, in the past tense, written for users rather than developers. Focus on what is now different
4. Output the entry in this format:
### [Category]
- [Single-line description in past tense]
If the change has no user-visible impact, say so plainly and recommend not adding a changelog entry.
```

> The library is now ten skills strong. The reader who has typed all ten into her `~/.claude/skills/` directory has, in roughly an hour of work, installed a set of capabilities that will pay her back many times over in every future Claude Code session. Each skill is small. None of them, alone, is impressive. But every one of them removes a small recurring friction, and the cumulative effect of ten removed frictions is substantial.

到这里,你的"库"就有了十个 skill。在大约一小时的工作里,把这十个都敲进 `~/.claude/skills/` 目录的读者,等于装下了一套"未来每个 Claude Code session 都会成倍回报"的能力集。每个 skill 都很小,单独看都不惊人;但每一个都消除了一个反复出现的小摩擦,而十个小摩擦被消除之后,累积效应是相当可观的。

Extending the Library · 扩展你的库

> A note on extension. The skills above are starting points, not endpoints. As the developer lives with them, she will notice that some of them fit her work perfectly and others fit imperfectly. The imperfect ones should be edited. A skill is a file. Editing a skill takes thirty seconds. A description that under-fires can have phrasings added. A body that produces output the developer does not like can have its instructions tightened. The library is meant to evolve. The reader who treats her `~/.claude/skills/` directory as a living project, revisited monthly and improved gradually, will find that within a year she has a personal toolkit that no off-the-shelf product could replicate, because it has been shaped by her actual work and her actual preferences.

关于"扩展"的几句话。上面这些 skill 是起点,不是终点。当开发者真正用上之后,会注意到:其中几个完全贴合自己的工作,有几个贴合得不够好。贴合得不够的就该改。skill 本质就是一个文件,改一个 skill 也就三十秒的事:触发不够稳的 description,加几句话;产出格式不满意的 body,把指令收紧。这个库天生就要进化。把自己的 `~/.claude/skills/` 目录当作一个"活项目"来对待——每月回头看一次,渐进

式改进——读者会发现,一年之后,她手里的个人工具包是任何现成产品都复刻不了的,因为它是被她真实的工作和真实的偏好"塑形"出来的。

← 前一页

后一页 →

> A second note on sharing. Several of the skills above will be useful to other developers in the reader's team or company. The mechanism for sharing is straightforward: copy the skill folder into the project's `.claude/skills/` directory and commit it. From that point forward, every developer who clones the project receives the skill automatically and benefits from it without any onboarding effort. This is how skill libraries spread within organizations. Not through documentation. Not through training sessions. Through configuration that travels with the code, and that the tool itself honors as soon as a colleague's session starts.

关于"分享"再说几句。上面有几个 skill,对读者所在团队或公司里的其他开发者也会很有用。分享机制很直接:把 skill 目录拷到项目的 `.claude/skills/` 下,提交进版本库即可。从那之后,凡是 clone 这个项目的开发者,都会自动拿到这些 skill,无需任何 onboarding 流程。这正是 skill 库在组织内传播的方式——不是靠文档,不是靠培训,而是靠"和代码一起走的配置":配置随代码一起传开,工具本身在同事一开 session 时就自动遵守它。

> A third note, on the deeper effect of having lived with a skill library for a few months. Developers who install ten skills and use them daily report a subtle shift in how they think about their own work. Tasks that used to feel like obligations, things they had to remember to do correctly, start to feel like artifacts that the environment produces on their behalf. The mental energy that used to be spent on remembering the commit format, on remembering the PR template, on remembering to run the right tests, gets freed up. That freed energy has somewhere to go, and where it goes, in most cases, is into the actual work the developer is being paid to do. The library does not merely save time. It frees attention. The attention is the more valuable of the two, and developers who internalize this distinction often find that within a few months they are doing work that would have been impossible for them at the start, not because they got smarter but because the environment got better at carrying everything that did not require their attention.

第三点,关于"和 skill 库共处几个月之后"那种更深层的效果。装了十个 skill 并日日使用的开发者,常会报告一种微妙的变化:他们看待自己工作的方式变了。原来那些"必须自己记得做对的事"——记 commit 格式、记 PR 模板、记跑对测试——开始变成"环境替我顺手产出的产物"。原本花在"记这些事"上的心力被释放出来了。这份释放出来的心力总要去一个地方;大多数情况下,它会去到那份"开发者真正被雇来做的"工作上。skill 库省下来的不只是时间,而是*注意力*。注意力比时间更值钱。把这个区别真正吃进去的开发者,常常会在几个月后发现自己能做的事,起点时根本做不到——不是因为变聪明了,而是因为环境把"那些不需要他们注意力的事"接管得越来越好了。

> Part Two ends here. The reader has, over six chapters, learned the anatomy of skills, built her first one by hand, mastered the description craft that determines whether skills fire correctly, learned to extend skills across multiple files when complexity demands it, internalized the principles that turn a collection of skills into a coherent library, and installed ten skills that will carry forward into every session ahead. Skills, the first pillar of the three-pillar framework, are now in working memory and in installed form. The next part of the book turns to the second pillar. Subagents are the parallel specialists that handle work which deserves its own context, its own role, and its own tool restrictions. The combination of skills and subagents is where the real architectural power of Claude Code begins to surface, and that surfacing begins in the chapter ahead.

第二部分到此收束。读者用六章的篇幅:学会了 skill 的解剖结构、手工搭出第一个 skill、掌握了"决定 skill 能不能正确触发"的 description 工艺、学会了在复杂度需要时把 skill 扩展到多文件、把"一堆 skill 变成一座连贯 library"的原则内化于心,以及装下十个会一路伴随今后每个 session 的 skill。"三柱框架"里的第一柱——Skills——已经既在肌肉记忆里,也在已安装的形态里。下一部分,书会转

向第二柱。Subagent 是并行运行的"专员",用来处理"应当拥有自己的上下文、自己的角色、自己的工具限制"的工作。Skills 和 subagent 组合起来,才是 Claude Code 真正架构力量开始显形的地方;这种显形,从下一章开始。

← 前一页 后一页 →

> *Editor's note: this page in the PDF source contains the remainder of two SKILL.md code blocks (decision-log and changelog-entry) that began on pages 85 and 86. The full blocks are already shown on those pages. The overflow content is reproduced below for completeness.*

编者注:PDF 源文件的本页包含两个 `SKILL.md` 代码块(decision-log 和 changelog-entry)的剩余部分——这两个代码块分别从第 85 页和第 86 页开始。完整的代码块已在前两页展示,这里为完整性再现溢出的部分。

> The tenth and final skill is `changelog-entry`. Its job is to write a single entry suitable for inclusion in a `CHANGELOG.md` file, given a recent change or set of changes.

第十个、也是最后一个 skill 是 `changelog-entry`。它的职责是:给定一组最近的改动,产出一条适合写入 `CHANGELOG.md` 的单条记录。

```
---
name: changelog-entry
description: Writes a single entry for a project's CHANGELOG.md based on a recent change. Use whenever the user asks
allowed-tools: Bash, Read
---
# Changelog Entry
When asked for a changelog entry, follow this procedure.
1. If the change is described in conversation, use that. If a commit or branch is referenced, read the diff to under
2. Categorize the change as one of: Added, Changed, Fixed, Removed, Deprecated, Security.
3. Write a single line, in the past tense, written for users rather than developers. Focus on what is now different
4. Output the entry in this format:
### [Category]
- [Single-line description in past tense]
If the change has no user-visible impact, say so plainly and recommend not adding a changelog entry.
```

← 前一页 后一页 →

> *Editor's note: this page in the PDF source contains the closing material of Chapter Nine (the changelog-entry SKILL.md block was overflowing from page 88). The closing paragraphs are also reproduced on page 87 for completeness; this page shows the SKILL.md overflow and the closing paragraph that was cut.*

编者注:PDF 源文件的本页包含第九章的收尾内容(changelog-entry 的 `SKILL.md` 代码块从第 88 页溢出)。结尾段落也已在第 87 页完整呈现;本页展示溢出的 `SKILL.md` 与被截断的段落。

> The library is now ten skills strong. The reader who has typed all ten into her `~/.claude/skills/` directory has, in roughly an hour of work, installed a set of capabilities that will pay her back many times over in every future Claude Code session. Each skill is small. None of them, alone, is impressive. But every one of them removes a small recurring friction, and the cumulative effect of ten removed frictions is substantial.

到这里,你的"库"就有了十个 skill。在大约一小时的工作里,把这十个都敲进 `~/.claude/skills/` 目录的读者,等于装下了一套"未来每个 Claude Code session 都会成倍回报"的能力集。每个 skill 都很小,单独看都不惊人;但每一个都消除了一个反复出现的小摩擦,而十个小摩擦被消除之后,累积效应是相当可观的。

Extending the Library · 扩展你的库

> A note on extension. The skills above are starting points, not endpoints. As the developer lives with them, she will notice that some of them fit her work perfectly and others fit imperfectly. The imperfect ones should be edited. A skill is a file. Editing a skill takes thirty seconds. A description that under-fires can have phrasings added. A body that produces output the developer does not like can have its instructions tightened. The library is meant to evolve. The reader who treats her `~/.claude/skills/` directory as a living project, revisited monthly and improved gradually, will find that within a year she has a personal toolkit that no off-the-shelf product could replicate, because it has been shaped by her actual work and her actual preferences.

关于"扩展"的几句话。上面这些 skill 是起点,不是终点。当开发者真正用上之后,会注意到:其中几个完全贴合自己的工作,有几个贴合得不够好。贴合得不够的就该改。skill 本质就是一个文件,改一个 skill 也就三十秒的事:触发不够稳的 description,加几句话;产出格式不满意的 body,把指令收紧。这个库天生就要进化。把自己的 `~/.claude/skills/` 目录当作一个"活项目"来对待——每月回头看一次,渐进式改进——读者会发现,一年之后,她手里的个人工具包是任何现成产品都复刻不了的,因为它是被她真实的工作和真实的偏好"塑形"出来的。

> A second note on sharing. Several of the skills above will be useful to other developers in the reader's team or company. The mechanism for sharing is straightforward: copy the skill folder into the project's `.claude/skills` directory and commit it. From that point forward, every developer who clones the project receives the skill automatically and benefits from it without any onboarding effort. This is how skill libraries spread within organizations. Not through documentation. Not through training sessions. Through configuration that travels with the code, and that the tool itself honors as soon as a colleague's session starts.

关于"分享"再说几句。上面有几个 skill,对读者所在团队或公司里的其他开发者也会很有用。分享机制很直接:把 skill 目录拷到项目的 `.claude/skills` 下,提交进版本库即可。从那之后,凡是 clone 这个项目的开发者,都会自动拿到这些 skill,无需任何 onboarding 流程。这正是 skill 库在组织内传播的方式——不是靠文档,不是靠培训,而是靠"和代码一起走的配置":配置随代码一起传开,工具本身在同事一开 session 时就自动遵守它。

← 前一页

后一页 →

> *Editor's note: this page in the PDF source contains the final paragraph of Chapter Nine (the closing of Part Two), which was also rendered on page 87*

> *because the SKILL.md code block over-flowed. It is reproduced here so the page is not empty in the PDF-aligned translation.*

编者注:PDF 源文件的本页包含第九章最后一段(也是第二部分的收束),这段内容也已在第 87 页渲染过——因为前面的 `SKILL.md` 代码块溢出。出于 PDF 页面对齐的考虑,这里再次呈现,避免该页在双语版里留白。

> A third note, on the deeper effect of having lived with a skill library for a few months. Developers who install ten skills and use them daily report a subtle shift in how they think about their own work. Tasks that used to feel like obligations, things they had to remember to do correctly, start to feel like artifacts that the environment produces on their behalf. The mental energy that used to be spent on remembering the commit format, on remembering the PR template, on remembering to run the right tests, gets freed up. That freed energy has somewhere to go, and where it goes, in most cases, is into the actual work the developer is being paid to do. The library does not merely save time. It frees attention. The attention is the more valuable of the two, and developers who internalize this distinction often find that within a few months they are doing work that would have been impossible for them at the start, not because they got smarter but because the environment got better at carrying everything that did not require their attention.

第三点,关于"和 skill 库共处几个月之后"那种更深层的效果。装了十个 skill 并日日使用的开发者,常会报告一种微妙的变化:他们看待自己工作的方式变了。原来那些"必须自己记得做对的事"——记 commit 格式、记 PR 模板、记跑对测试——开始变成"环境替我顺手产出的产物"。原本花在"记这些事"上的心力被释放出来了。这份释放出来的心力总要去一个地方;大多数情况下,它会去到那份"开发者真正被雇来做的"工作上。skill 库省下来的不只是时间,而是*注意力*。注意力比时间更值钱。把这个区别真正吃进去的开发者,常常会在几个月后发现自己能做的事,起点时根本做不到——不是因为变聪明了,而是因为环境把"那些不需要他们注意力的事"接管得越来越好了。

> Part Two ends here. The reader has, over six chapters, learned the anatomy of skills, built her first one by hand, mastered the description craft that determines whether skills fire correctly, learned to extend skills across multiple files when complexity demands it, internalized the principles that turn a collection of skills into a coherent library, and installed ten skills that will carry forward into every session ahead. Skills, the first pillar of the three-pillar framework, are now in working memory and in installed form. The next part of the book turns to the second pillar. Subagents are the parallel specialists that handle work which deserves its own context, its own role, and its own tool restrictions. The combination of skills and subagents is where the real architectural power of Claude Code begins to surface, and that surfacing begins in the chapter ahead.

第二部分到此收束。读者用六章的篇幅:学会了 skill 的解剖结构、手工搭出第一个 skill、掌握了"决定 skill 能不能正确触发"的 description 工艺、学会了在复杂度需要时把 skill 扩展到多文件、把"一堆 skill 变成一座连贯 library"的原则内化于心,以及装下十个会一路伴随今后每个 session 的 skill。"三柱框架"里的第一柱——Skills——已经既在肌肉记忆里,也在已安装的形态里。下一部分,书会转向第二柱。Subagent 是并行运行的"专员",用来处理"应当拥有自己的上下文、自己的角色、自己的工具限制"的工作。Skills 和 subagent 组合起来,才是 Claude Code 真正架构力量开始显形的地方;这种显形,从下一章开始。